

HavenWorld Build GuidePart 5B — Phase 2

*3D Avatars, Blender Pipeline, Furniture System, and Personal Rooms
Rigged Avatar GLB Production, Mixamo Animations, Morph-Target
Customizer, Furniture Placement, and Player Room Editor*

Series: HavenWorld Developer Build Guide | **Part:** 5B | **Phase:** 2 — 3D Asset Pipeline

Author: Daniel | **Date:** Friday, 18 September 2026 | **License:** GPL-3.0 | **Tier:** Free only
(no paid services)

Table of Contents

1. **1.** Part 5B Overview and What You'll Build
2. **2.** Blender Pipeline Overview
3. 2.1 Why GLB (Binary glTF)?
4. 2.2 Blender Scene Setup Rules for HavenWorld Assets
5. 2.3 Room Asset Creation Walkthrough
6. 2.4 Verifying Assets in the Babylon.js Sandbox
7. **3.** The Base Avatar GLB: Rigged Humanoid
8. 3.1 Obtaining a Free Rigged Base Character
9. 3.2 Adapting the Mixamo Character for HavenWorld
- 10.3.3 Adding Morph Targets for Avatar Customization
- 11.3.4 Avatar Material Slots and Color Customization
- 12.3.5 Exporting the Base Avatar as GLB
- 13.**4.** Mixamo Animations Integration
- 14.4.1 Downloading Animations from Mixamo

- 15.4.2 Merging Animations into the Avatar GLB with Blender
- 16.4.3 Playing Animations in Babylon.js
- 17.**5.** Complete AvatarController.ts with Real GLB
- 18.**6.** 3D Avatar Customizer UI
 - 19.6.1 AvatarCustomizer.ts — The In-Game Customizer Screen
 - 20.6.2 Server Route Update — Save Avatar Customization
- 21.**7.** Furniture System
 - 22.7.1 Furniture Data Model Review
 - 23.7.2 FurnitureManager.ts — Loading and Placing Furniture
 - 24.7.3 Creating Placeholder Furniture GLBs
- 25.**8.** Personal Room Editor
 - 26.8.1 Overview of the Room Editor
 - 27.8.2 RoomEditor.ts — Complete Implementation
 - 28.8.3 Server Route — Save Room Layout
 - 29.8.4 Socket Events for Real-Time Furniture Sync
- 30.**9.** Seed Data: Default Furniture Items
- 31.**10.** Performance Optimization for 3D Browser Games
 - 32.10.1 Asset Loading Strategy
 - 33.10.2 Texture Atlasing for Furniture
 - 34.10.3 Level of Detail (LOD) Strategy
 - 35.10.4 Render Loop Performance Budget
- 36.**11.** Complete Phase 2 Testing Checklist
- 37.**12.** Git Workflow and Deployment
 - 38.12.1 Committing Phase 2
 - 39.12.2 Cloudflare Pages Deployment Check
 - 40.12.3 Oracle VM Server Updates

- 41. **13.** Troubleshooting Phase 2B
 - 42. **14.** What's Next: Phase 3 Preview
 - 43. **App.** Quick Reference Cards
-

1. Part 5B Overview and What You'll Build

Part 5A established the Babylon.js engine foundation for HavenWorld: a working render loop, a `SceneManager` that loads and unloads scenes, a basic `HavenEngine` wrapper, and placeholder geometry — a capsule mesh standing in for the local player avatar and flat colored planes standing in for room floors and walls. Those placeholders were intentional scaffolding so you could verify the engine wiring before committing to an art pipeline. Part 5B is where the scaffolding comes down and production-quality 3D assets take its place.

By the end of this guide you will have built six interconnected systems that together constitute the entirety of Phase 2's visual and interactive layer. Each system builds on the previous one, so the sections are meant to be followed in order on your first pass. After that you can revisit individual sections as reference material.

The six deliverables are:

- 44. **Blender → GLB export pipeline** — standardized rules, coordinate-system handling, UV unwrapping, texture compression with `glTF-transform`, and sandbox verification. Every room and avatar asset in HavenWorld will go through this pipeline.
- 45. **Rigged humanoid base avatar** — a low-poly (under 8,000-triangle) character with a proper bone hierarchy, imported from Mixamo, cleaned up in Blender, and exported as a single `base_avatar.glb` that contains the mesh, armature, shape keys (morph targets), and all five animations.
- 46. **Mixamo animation integration** — idle, walk, sit, wave, and dance animations downloaded from Mixamo (free tier), merged into the avatar GLB via Blender's NLA Editor, and driven in-engine by a typed animation manager inside `AvatarController.ts`.

47. **In-game 3D avatar customizer** — a full-screen overlay UI with a live 3D preview, sliders that drive morph-target blending for body shape, color pickers for skin/hair/eyes/clothing, and a `PUT /api/users/me/avatar` endpoint that persists the result and immediately broadcasts it to other players via Socket.io.
48. **Furniture placement system** — a `FurnitureManager` that fetches a room's furniture layout from the server and instances each item's GLB at its saved position and rotation, with a placeholder GLB generator so you have real assets on day one.
49. **Personal room editor** — a drag-place-rotate-save editor that activates when a player enters their own room, shows a snapping floor grid, lets the player reposition furniture, and saves the entire layout to the server in a single transaction that triggers a real-time update for all players in the room.

Prerequisites — Confirm Before Proceeding

- **Part 5A complete** — `HavenEngine`, `SceneManager`, `RoomScene`, and `LobbyScene` are working and deployed.
- **Blender 5.1 installed** — available free at blender.org. glTF 2.0 export is built in; no add-ons required for this guide.
- **Free Adobe account** — required to use Mixamo (mixamo.com). Adobe accounts are free; no Creative Cloud subscription needed for Mixamo access.
- **gltf-transform CLI installed** — run `npm install -g @gltf-transform/cli` once globally. Used for asset inspection and texture compression.
- **Node.js 20+ and pnpm 9+** — already required from Part 4A.
- **PostgreSQL running locally** — Prisma schema from Part 4A is migrated and seeded.

Estimated time for this entire guide: **3-5 weekends** for a solo developer already comfortable with TypeScript. Blender work (Sections 2-4) takes the most time on a first pass; the TypeScript implementation (Sections 5-8) is faster once the assets are ready.

2. Blender Pipeline Overview

2.1 Why GLB (Binary glTF)?

GLB (GL Transmission Format, Binary) is the binary container form of the **glTF 2.0** specification. Where a standard `.gltf` file is a JSON descriptor that references external `.bin` buffers and texture images as separate files, a `.glb` packs all of that — geometry, animation data, material parameters, and embedded textures — into a single binary file. For a browser-based game this is a significant practical advantage: one HTTP request instead of many, predictable loading times, and no risk of missing texture files breaking a model.

The glTF 2.0 specification covers every feature HavenWorld needs: **Physically Based Rendering (PBR)** materials with Base Color, Roughness, Metallic, Normal, Emissive, and Occlusion channels; **skeletal animation** (skinning) with joint matrices; **morph targets** (called Shape Keys in Blender) for blend-shape customization; and multiple animation clips stored in a single asset. All of these features are first-class in the standard — not vendor extensions — which means any compliant loader can read them.

Babylon.js v7 ships with a native glTF 2.0 loader (`@babylonjs/loaders`) that requires no additional plugins. The loader automatically converts the glTF Y-up coordinate system to Babylon's Y-up world, parses all animation clips into `AnimationGroup` objects, and exposes morph targets through a `MorphTargetManager` on each mesh. You do not need to write any custom parsing code.

Blender 5.1 includes a built-in glTF 2.0 exporter (no add-on required) that has been maintained in sync with the spec for several years. It handles the Z-up to Y-up coordinate conversion automatically during export, so you can model in Blender's native coordinate system without manual rotation hacks.

2.2 Blender Scene Setup Rules for HavenWorld Assets

Every Blender file that produces a HavenWorld asset must follow these rules without exception. They exist to prevent the most common class of export bugs: wrong scale, broken coordinate system, invisible materials, missing UVs, and naming collisions in the Babylon.js scene graph. Save this list as a checklist and run through it before every export.

Rule #	Setting / Action	Where in Blender	Rationale
1	Units: Metric, Unit Scale = 1.0	Scene Properties → Units	1 Blender unit = 1 meter = 1 Babylon.js world unit. Mismatched scale causes avatars to be 100× too large or too small.
2	Apply All Transforms before export (Ctrl+A → All Transforms)	Object Mode, with mesh selected	Un-applied transforms bake as offsets in glTF nodes, causing mesh pivot and scale bugs in-engine.
3	Coordinate system: Z-up (Blender native). Do not rotate manually to Y-up.	N/A — handled by exporter	The glTF exporter applies the Z-up → Y-up conversion automatically. Manual rotation doubles it.
4	Materials: Principled BSDF shader only. No custom node groups.	Shader Editor	Only Principled BSDF maps cleanly to glTF PBR. Custom node groups are silently dropped on export.
5	Texture slots: Base Color, Roughness, Metallic, Normal Map, Emission only	Principled BSDF inputs	These are the five standard glTF PBR texture slots. Other inputs are not exported.

Rule #	Setting / Action	Where in Blender	Rationale
6	Texture resolution: 512×512 avatar parts, 1024×1024 furniture, 2048×2048 rooms	Image Editor / UV Editor	Higher resolutions increase GLB file size with diminishing visual returns at web scale.
7	Polygon budget: Rooms <50,000 tris, Avatars <8,000 tris, Furniture <2,000 tris each	Mesh Properties → Statistics overlay	Keeping within budget allows 12 simultaneous players plus full room geometry under 200,000 total triangles.
8	Mesh naming: snake_case. Collision prefix: col_. NavMesh: navmesh_floor. Walkable: walkable_*. Furniture anchors: anchor_	Outliner (F2 to rename)	The TypeScript scene code queries meshes by name. Consistent naming avoids runtime lookup failures.
9	Armature named Armature. Root bone named Root. Spine chain: Root > Hips > Spine > Spine1 > Spine2 > Neck > Head	Outliner / Armature Properties	AvatarController.ts looks up the Head bone by name to position name tags.
10	UV unwrap all meshes before export	Edit Mode: A → U → Smart UV Project	Meshes without UVs cause the glTF exporter to emit a warning and materials may render black in-engine.

2.3 Room Asset Creation Walkthrough

This walkthrough creates the first public room: **Town Square**. It is a 20×20-meter open plaza with walls, a placeholder fountain, and a navigable floor. Follow every step in sequence; the naming conventions established here are consumed directly by the TypeScript room loader.

50. Launch Blender 5.1. In the default scene, select the default cube (left-click), then press **X** and confirm **Delete**.
51. Open **Scene Properties** (camera icon in the Properties panel). Under **Units**, set *Unit System* to **Metric** and *Unit Scale* to **1.0**.
52. Press **Shift+A → Mesh → Plane** to add a ground plane. With it selected, press **S**, then type **10**, then **Enter**. This scales the 2-meter default plane to 20×20 meters.
53. Press **F2** in the Outliner to rename the plane to `walkable_floor`.
54. Add the four walls. For each: **Shift+A → Mesh → Plane**, then in the sidebar (**N**) set Dimensions to 20m × 4m. Rotate 90° on the X axis (**R, X, 90, Enter**). Translate to the correct edge. Name them `wall_north`, `wall_south`, `wall_east`, `wall_west`.
55. Add decorative props. For a bench: **Shift+A → Mesh → Cube**, scale to roughly 1.5m × 0.4m × 0.45m, name it `prop_bench_01`. For a fountain placeholder: a cube scaled to 2m × 2m × 0.5m, named `prop_fountain`. Position both in the plaza.
56. Create the NavMesh. Select `walkable_floor`, press **Shift+D** then **Enter** to duplicate in place. Name the duplicate `navmesh_floor`. In the Outliner, create a new Collection named **NavMesh** and move `navmesh_floor` into it. This mesh will be invisible at runtime but used by the pathfinding logic.
57. Apply materials. Select each mesh, open the **Material Properties** tab, click **+** to add a new material, rename it (e.g., `mat_floor`), and ensure the shader is **Principled BSDF**. Assign a Base Color. Repeat for each mesh.
58. UV unwrap all meshes at once: press **A** to select all in Object Mode, then enter Edit Mode (**Tab**), press **A** to select all faces, then **U → Smart UV Project**. Accept defaults. Return to Object Mode (**Tab**).
59. Apply all transforms: press **A** to select all objects, then **Ctrl+A → All Transforms**.

60. Export: **File** → **Export** → **glTF 2.0**. In the export panel: *Format* = **GLB**. Under *Include*: Mesh ON, Materials ON, Apply Modifiers ON, Draco compression OFF (we compress with gltf-transform separately). Navigate to your project root and save as `public/assets/rooms/town_square.glb`.

After exporting, compress and optimize with `gltf-transform`:

```
npx @gltf-transform/cli optimize public/assets/rooms/town_square.glb \
public/assets/rooms/town_square.glb \ --texture-compress webp
```

Note: WebP Texture Compression

The `--texture-compress webp` flag re-encodes all embedded textures as WebP. WebP textures are supported by all modern browsers (Chrome, Firefox, Safari 14+, Edge) and typically reduce GLB file size by 40–70% compared to PNG. Always run this step before committing assets to the repository.

2.4 Verifying Assets in the Babylon.js Sandbox

Before writing a single line of TypeScript to load a new GLB, drag the file into the **Babylon.js Sandbox** at `sandbox.babylonjs.com`. The sandbox uses the same Babylon.js version as your project and will expose every export issue — wrong scale, missing materials, broken armature hierarchy, missing animations — in seconds without any code. This is the single fastest debugging step in the entire asset pipeline and should become a habit.

In the sandbox, check: (1) the mesh loads at a sane scale relative to the ground grid; (2) all materials display with correct colors (not magenta “missing material” fallback); (3) the Inspector panel on the left shows all expected mesh names matching your naming conventions; (4) the **Animation Groups** section lists the expected animation names with correct durations. If any check fails, fix it in Blender and re-export before proceeding to the TypeScript integration.

3. The Base Avatar GLB: Rigged Humanoid

3.1 Obtaining a Free Rigged Base Character

A fully rigged humanoid character — clean topology, proper bone weights, industry-standard joint hierarchy — represents weeks of skilled 3D art work if built from scratch. HavenWorld takes the pragmatic path: start with a free, pre-rigged base mesh and customize it for the project's requirements. There are two realistic options for a GPL-3.0 project with no budget.

Option A — Mixamo Y Bot or X Bot (Recommended for this guide): Adobe's Mixamo service hosts two purpose-built, low-poly rigged characters called **Y Bot** and **X Bot**. Both are clean humanoid meshes with a standard 65-bone hierarchy, proper skin weights, and a neutral T-pose. They are available for download at no cost with a free Adobe account. Their geometry is intentionally featureless (robot-like), which actually works well as a base for a customizable avatar system since you will be adding morph-target shape variation and material-based color customization on top. The download format is FBX, which imports directly into Blender.

Option B — Blender Rigify from Scratch (Advanced, not covered in this part): Blender's built-in **Rigify** add-on can auto-generate a full humanoid control rig from a metadata rig template. This gives you complete creative control over the mesh topology and bone structure but requires substantial Blender proficiency and is a multi-weekend project on its own. This path is documented in the HavenWorld Advanced Art Guide (a future document in this series). For Part 5B, use Option A.

To download the Y Bot from Mixamo:

61. Navigate to mixamo.com and sign in with your free Adobe account.
62. Click the **Characters** tab. Search for **Y Bot**.
63. Click Y Bot to select it. On the right panel, click **Download**.
64. In the Download dialog: Format = **FBX**, Pose = **T-pose**. Click **Download**.
Save the file as `source_assets/mixamo_ybot_tpose.fbx`.

3.2 Adapting the Mixamo Character for HavenWorld

The Mixamo FBX comes with a naming convention that prefixes every bone name with `mixamorig:` (for example, `mixamorig:Hips`, `mixamorig:Spine`). While functionally correct, these prefixed names create verbose and fragile lookups in TypeScript code. The first step is to strip the prefix and rename the armature to match HavenWorld's required naming convention from Rule 9 in Section 2.2.

After importing the FBX into Blender (**File → Import → FBX**, accept all default import settings), open the **Scripting** workspace tab and run the following Python script in Blender's built-in Python console to batch rename all bones:

```
import bpy # Target the imported armature – rename it first
armature_obj = bpy.data.objects.get('Armature')
if armature_obj is None: # Mixamo sometimes imports as 'Armature' or the character name
    for obj in bpy.data.objects:
        if obj.type == 'ARMATURE':
            armature_obj = obj
            armature_obj.name = 'Armature'
armature_obj.data.name = 'Armature'
break
if armature_obj:
    for bone in armature_obj.data.bones:
        if 'mixamorig:' in bone.name:
            bone.name = bone.name.replace('mixamorig:', '')
            print(f"Renamed {len(armature_obj.data.bones)} bones successfully.")
        else:
            print("ERROR: No armature found in scene.")
```

Run the script by clicking the **Run Script** button (the triangle/play icon) in the Scripting workspace. After running, switch to Pose Mode (**Ctrl+Tab**) and verify that bones are now named `Hips`, `Spine`, `Head`, etc. without the prefix.

Next, scale the character to 1.7 meters in height. Select the armature in Object Mode and check its current Z-dimension in the **N panel → Item → Dimensions**. Adjust the Z scale so the total height reads approximately 1.7m. Then apply transforms to both the mesh and the armature separately:

```
# In Blender Python console:
import bpy
bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.transform_apply(location=True, rotation=True, scale=True)
print("Transforms applied to all objects.")
```

3.3 Adding Morph Targets for Avatar Customization

HavenWorld's avatar customizer gives players control over body shape through **morph targets**, called **Shape Keys** in Blender. A shape key is a stored deformation of the mesh relative to the Basis (default) shape. In the exported GLB, these become glTF morph targets with influence weights ranging from 0.0 (no effect) to 1.0 (full deformation). In Babylon.js, they are accessible through each mesh's `MorphTargetManager`.

HavenWorld defines six shape keys for the base avatar. Add them to the body mesh (not the armature) as follows. Select the body mesh in Object Mode, then open **Properties → Object Data Properties → Shape Keys**:

Shape Key Name	Description	Editing Technique	AvatarData field
Basis	Default T-pose shape. Created automatically when you add the first key.	No edits needed.	N/A (always present)
fat	Increased torso width, belly protrusion, wider hips.	Proportional editing (O) on torso vertices; falloff = Smooth.	bodyType (0.0→1.0)
thin	Reduced body mass, narrower shoulders and hips.	Scale torso inward with proportional editing.	bodyType at negative values mapped to this key
tall	Stretched torso and legs upward.	Select all vertices from hips up; scale on Z axis.	height (0.5→1.0)
short	Compressed torso and legs.	Select all vertices from hips up; scale inward on Z.	height (0.0→0.5)
muscular	Broader shoulders, larger arm and chest definition.	Proportional editing on upper arms and chest.	build (0.0→1.0)

For each shape key after `Basis`: (1) Click the **+** button in Shape Keys to add a new key. (2) Double-click the key name in the list to rename it (e.g., `fat`). (3) With that key **selected (highlighted in blue)** in the list, enter Edit Mode. *Only when the non-Basis key is selected do your edits affect that shape key's deformation.* (4) Use Proportional Editing (**O** key) and standard move/scale tools to sculpt the desired shape. (5) Return to Object Mode. The shape key is saved.

You can preview the result by dragging the **Value** slider (0.0–1.0) next to each shape key name in Object Mode. Setting `fat` to 0.8 and `muscular` to 0.5 simultaneously should produce an additive blend of both deformations. This is exactly how `AvatarController.ts` will drive them at runtime.

3.4 Avatar Material Slots and Color Customization

Runtime color customization in HavenWorld works by targeting named material slots on the avatar mesh and changing their `albedoColor` property (for PBR materials) at runtime in Babylon.js. This requires each customizable region of the avatar to have its own dedicated material slot. Set up five material slots on the body mesh in Blender:

65.**mat_skin** — Applied to all exposed skin faces (body, face, hands, neck). The Base Color of this material is the runtime skin tone.

66.**mat_hair** — Applied to the hair mesh (or hair polygon region if modeled as part of the same mesh). Controls hair color.

67.**mat_eyes** — Applied to the eye sphere meshes. Controls eye color. Use a slightly emissive material for a subtle glow effect.

68.**mat_shirt** — Applied to the torso clothing region. Controls top color in Phase 2 (full clothing items arrive in Phase 3).

69.**mat_pants** — Applied to the leg clothing region. Controls bottom color.

To add multiple material slots in Blender: In the Material Properties panel, click **+** to add each slot, name it, then in Edit Mode select the faces for that region and click **Assign** to assign those faces to the active slot. Repeat for each of the five slots.

At runtime in Babylon.js, the AvatarController accesses these by material name after loading the GLB:

```
// Example: change skin tone at runtime const skinMesh =  
scene.getMeshByName('Body'); // the body sub-mesh if (skinMesh &&  
skinMesh.material) { const mat = skinMesh.material as BABYLON.PBRMaterial;  
mat.albedoColor = BABYLON.Color3.FromHexString('#F5CBA7'); }
```

Because glTF materials are named in the GLB file, you can also find them via `scene.getMaterialByName('mat_skin')` if Babylon creates them as shared materials rather than per-mesh materials. Both approaches work; querying by material name is more robust.

3.5 Exporting the Base Avatar as GLB

Once the character has all shape keys, material slots, and properly named bones, export it with these exact settings to ensure the GLB contains everything needed:

70. In Object Mode, select BOTH the body mesh AND the armature (Shift+click each).

71. Go to **File → Export → glTF 2.0**.

72. In the export dialog, configure these settings:

Export Category	Setting	Value
Format	Format	GLB (.glb)
Include	Limit to	Selected Objects ONLY
Mesh	Apply Modifiers	ON
Mesh	UVs	ON
Mesh	Normals	ON
Mesh	Vertex Colors	OFF (not used)
Mesh	Shape Keys	ON
Armature	Export Deformation Bones Only	ON (removes Rigify control bones if present)
Skinning	Skinning	ON
Animation	Animation	OFF for this export (added in Section 4)
Material	Materials	Export (standard)
Compression	Draco	OFF (use gltf-transform separately)

Save the file as `public/assets/avatars/base_avatar.glb`.

Verify and optimize the export:

```
npx @gltf-transform/cli inspect public/assets/avatars/base_avatar.glb
```

The inspect output should show: 1 mesh, correct triangle count (under 8,000), 6 morph targets (Basis + 5 shape keys), 1 skin, 5 material slots. If any of these are missing, return to Blender and fix the export settings.

```
npx @gltf-transform/cli optimize public/assets/avatars/base_avatar.glb \
public/assets/avatars/base_avatar.glb \ --texture-compress webp
```

4. Mixamo Animations Integration

4.1 Downloading Animations from Mixamo

HavenWorld needs exactly five core animations for Phase 2. All five are available on Mixamo at no cost. The key requirement when downloading is to use **Without Skin** (Skin = Without Skin in the download dialog) because you are retargeting the animation onto your own cleaned-up skeleton, not the default Mixamo skeleton. Using “With Skin” would embed a second copy of the mesh inside each animation file, wasting disk space and complicating the merge step.

Anim Name	Mixamo Search Term	In Place	FPS	Save As
idle	"Breathing Idle" or "Standing Idle"	N/A	30	source_assets/anim/idle.fbx
walk	"Walking"	ON	30	source_assets/anim/walk.fbx
sit	"Sitting Idle"	N/A	30	source_assets/anim/sit.fbx
wave	"Waving Gesture" or "Wave Hip Hop Dance"	N/A	30	source_assets/anim/wave.fbx
dance	"Samba Dancing" or "Gangnam Style"	N/A	30	source_assets/anim/dance.fbx

For each animation: (1) On Mixamo, click **Characters** and ensure your Y Bot character is selected. (2) Click **Animations** and search for the term above. (3) Adjust the animation preview on the character. For walk: enable **In Place** toggle. (4) Click **Download**: Format = **FBX Binary (.fbx)**, Skin = **Without Skin**, Frames per Second = **30**, Keyframe Reduction = **uniform**. (5) Download and save to the path shown above.

4.2 Merging Animations into the Avatar GLB with Blender

Each Mixamo FBX contains a single animation action on a copy of the armature. The goal is to import all five animation actions into the base avatar's armature and organize them as NLA (Non-Linear Animation) strips, then export a final GLB that contains the mesh, skeleton, shape keys, and all five animations as separate `AnimationGroup` objects readable by Babylon.js.

73. Open your `base_avatar.blend` file (the file with the cleaned armature and shape keys). **Do not open a new Blender instance** — all work happens in this file.
74. Import `idle.fbx`: **File → Import → FBX**. A second armature object named something like `Armature.001` will appear in the scene with the idle animation data.
75. Switch to the **Action Editor** (Window → New Editor Type → Action Editor, or find it in the Animation workspace). On the newly imported armature (`Armature.001`), locate the action in the Action Editor dropdown. Click the action name and rename it to exactly `idle` (all lowercase — this is the name Babylon.js will expose as the AnimationGroup name).
76. Switch to the **NLA Editor**. Select your *original* base armature (`Armature`, not the imported one). In the NLA Editor, click **Push Down** on the idle action strip to create a permanent NLA strip. Rename the strip to `idle`.
77. Delete the imported `Armature.001` object — you only needed it to transfer the animation action. Select it, press **X → Delete**.
78. Repeat steps 2–5 for each remaining animation: `walk.fbx`, `sit.fbx`, `wave.fbx`, `dance.fbx`. After all five are done, the base armature's NLA Editor should show five strips named `idle`, `walk`, `sit`, `wave`, `dance`.
79. Final export: **File → Export → glTF 2.0**. Same settings as Section 3.5, but now enable: **Animation = ON, NLA Strips = ON, Group by NLA Track = ON**. Export to `public/assets/avatars/base_avatar.glb` (overwrite the previous version).

Drag the new GLB into the Babylon.js Sandbox. In the Inspector, open **Animation Groups**. You should see exactly five groups: `idle`, `walk`, `sit`, `wave`, `dance`. Click each to preview it. If any group is missing or named incorrectly, return to Blender and verify the NLA strip names.

4.3 Playing Animations in Babylon.js

The following code belongs in `AvatarController.ts` and handles the full lifecycle of animation playback including basic cross-fade blending. Babylon.js `AnimationGroup` objects support weight-based blending through `setWeightForAllAnimatables()`, which allows smooth transitions between states by ramping the outgoing animation's weight to 0 while ramping the incoming animation's weight to 1 over a set number of frames.

```
// — Animation Management (add to AvatarController class)
private animations: Map<string, BABYLON.AnimationGroup>
= new Map(); private currentAnim: BABYLON.AnimationGroup | null = null;
private blendTimer: number | null = null; private readonly BLEND_FRAMES = 9;
// ~0.3s at 30fps private initAnimations(animationGroups:
BABYLON.AnimationGroup[]): void { const validNames = ['idle', 'walk',
'sit', 'wave', 'dance']; for (const group of animationGroups) { const
name = group.name.toLowerCase().trim(); if (validNames.includes(name)) {
group.stop(); group.setWeightForAllAnimatables(0);
this.animations.set(name, group); } } // Start all animation groups
simultaneously at weight 0 except idle // Babylon blending requires all
groups to be playing this.animations.forEach((group) => group.play(true));
this.crossFadeTo('idle'); } playAnim(name: string, loop = true): void
{ this.crossFadeTo(name, loop); } private crossFadeTo(name: string, loop =
true): void { const incoming = this.animations.get(name); if (!incoming)
{ console.warn(`[AvatarController] Animation not found: ${name}`);
return; } const outgoing = this.currentAnim; this.currentAnim =
incoming; if (this.blendTimer !== null)
{ clearInterval(this.blendTimer); } let frame = 0; this.blendTimer
= window.setInterval(() => { const progress = Math.min(frame /
this.BLEND_FRAMES, 1); incoming.setWeightForAllAnimatables(progress);
if (outgoing && outgoing !== incoming)
{ outgoing.setWeightForAllAnimatables(1 - progress); } frame++;
if (frame > this.BLEND_FRAMES) { clearInterval(this.blendTimer!);
this.blendTimer = null; // Stop outgoing animation to reduce sampler
overhead if (outgoing && outgoing !== incoming)
{ outgoing.setWeightForAllAnimatables(0); } } }, 1000 /
30); // tick at 30fps }
```

5. Complete AvatarController.ts with Real GLB

This replaces the Part 5A placeholder capsule implementation entirely. The file lives at `apps/client/src/world/AvatarController.ts`. It handles loading the avatar GLB, wiring up animations and morph targets, rendering the name tag, and exposing the movement API to the scene layer.

```
// apps/client/src/world/AvatarController.ts import * as BABYLON from
'@babylonjs/core'; import { AdvancedDynamicTexture, Rectangle, TextBlock }
from '@babylonjs/gui'; import type { AvatarData } from
'../ui/AvatarCustomizer'; const AVATAR_GLB_PATH = '/assets/avatars/'; const
AVATAR_GLB_FILE = 'base_avatar.glb'; const WALK_SPEED = 3.5; // meters per
second const ARRIVAL_THRESHOLD = 0.15; // stop walking when within 15cm
export class AvatarController { private scene: BABYLON.Scene; private
rootMesh: BABYLON.AbstractMesh | null = null; private skeleton:
BABYLON.Skeleton | null = null; private morphManagers:
BABYLON.MorphTargetManager[] = []; private animations: Map<string,
BABYLON.AnimationGroup> = new Map(); private currentAnim:
BABYLON.AnimationGroup | null = null; private blendTimer: number | null =
null; private readonly BLEND_FRAMES = 9; private nameTagTexture:
AdvancedDynamicTexture | null = null; private displayName: string;
private targetPosition: BABYLON.Vector3 | null = null; private isMoving =
false; // Expose position for RemoteAvatar sync public get position():
BABYLON.Vector3 { return this.rootMesh?.position ??
BABYLON.Vector3.Zero(); } constructor(scene: BABYLON.Scene, displayName:
string) { this.scene = scene; this.displayName =
displayName; } // — Public API

load(spawnPosition: BABYLON.Vector3): Promise<void> { async
const result =
await BABYLON.SceneLoader.ImportMeshAsync(' ', AVATAR_GLB_PATH,
AVATAR_GLB_FILE, this.scene ); if (!result.meshes.length) {
throw new Error('[AvatarController] GLB loaded but contained no meshes. ');
} // The first mesh in a GLB import is always the root transform node
this.rootMesh = result.meshes[0]; this.rootMesh.name = `avatar_local`;
this.rootMesh.position = spawnPosition.clone(); // Collect morph target
managers from all sub-meshes for (const mesh of result.meshes) { if
(mesh.morphTargetManager)
{ this.morphManagers.push(mesh.morphTargetManager); } }
// Babylon imports GLB skeletons separately if (result.skeletons.length >
0) { this.skeleton = result.skeletons[0]; }
this.initAnimations(result.animationGroups); this.attachNameTag();
this.registerUpdateLoop(); console.log(`[AvatarController]
Loaded. Meshes: ${result.meshes.length}, ` + `MorphManagers: $
{this.morphManagers.length}, ` + `AnimGroups: $
{result.animationGroups.length}` ); } moveTo(target:
BABYLON.Vector3): void { this.targetPosition = new
BABYLON.Vector3(target.x, this.rootMesh?.position.y ?? 0, target.z); if
(!this.isMoving) { this.isMoving = true;
this.crossFadeTo('walk'); } } playSocialAnim(name: 'sit' | 'wave' |
'dance'): void { this.targetPosition = null; this.isMoving = false;
this.crossFadeTo(name, name !== 'wave'); // wave does not loop }
applyCustomization(data: AvatarData): void
{ this.applyMorphTargets(data); this.applyMaterialColors(data); }
dispose(): void { if (this.blendTimer !== null)
```

```

clearInterval(this.blendTimer);    this.nameTagTexture?.dispose();
this.rootMesh?.dispose();    }    // — Morph Targets

private
applyMorphTargets(data: AvatarData): void {    for (const manager of
this.morphManagers) {    for (let i = 0; i < manager.numTargets; i++) {
const target = manager.getTarget(i);    switch
(target.name.toLowerCase()) {    case 'fat':
target.influence = Math.max(0, data.bodyType);    break;
case 'thin':    target.influence = Math.max(0, -data.bodyType + 0.5)
* 2;    break;    case 'tall':    target.influence =
Math.max(0, (data.height - 0.5) * 2);    break;    case
'short':    target.influence = Math.max(0, (0.5 - data.height) * 2);
break;    case 'muscular':    target.influence = data.build;
break;    }    }    }    }    private applyMaterialColors(data:
AvatarData): void {    const colorMap: Record<string, string> =
{    mat_skin: data.skinTone,    mat_hair: data.hairColor,
mat_eyes: data.eyeColor,    mat_shirt: data.topColor,    mat_pants:
data.bottomColor,    };    for (const [matName, hexColor] of
Object.entries(colorMap)) {    const mat =
this.scene.getMaterialByName(matName) as BABYLON.PBRMaterial | null;    if
(mat) {    mat.albedoColor = BABYLON.Color3.FromHexString(hexColor);
}    }    }    // — Name Tag

private
attachNameTag(): void {    if (!this.rootMesh) return;    const plane =
BABYLON.MeshBuilder.CreatePlane(`nametag_${this.displayName}`,    {
width: 1.6, height: 0.4 },    this.scene    );    plane.parent =
this.rootMesh;    plane.position = new BABYLON.Vector3(0, 2.1, 0); // above
head    plane.billboardMode = BABYLON.Mesh.BILLBOARDMODE_ALL;
plane.isPickable = false;    this.nameTagTexture =
AdvancedDynamicTexture.CreateForMesh(plane, 512, 128);    const bg = new
Rectangle();    bg.background = '#00000099';    bg.cornerRadius = 8;
bg.thickness = 0;    bg.width = '100%';    bg.height = '100%';
this.nameTagTexture.addControl(bg);    const label = new TextBlock();
label.text = this.displayName;    label.color = '#ffffff';
label.fontSize = 32;    label.fontFamily = 'Calibri';
bg.addControl(label);    }    // — Animation System

private
initAnimations(animationGroups: BABYLON.AnimationGroup[]): void {    const
validNames = ['idle', 'walk', 'sit', 'wave', 'dance'];    for (const group
of animationGroups) {    const name = group.name.toLowerCase().trim();
if (validNames.includes(name)) {    group.stop();
group.setWeightForAllAnimatables(0);    this.animations.set(name,
group);    }    }    this.animations.forEach((group) =>
group.play(true));    this.crossFadeTo('idle');    }    private
crossFadeTo(name: string, loop = true): void {    const incoming =
this.animations.get(name);    if (!incoming) return;    const outgoing =
this.currentAnim;    this.currentAnim = incoming;    if (this.blendTimer !
== null) clearInterval(this.blendTimer);    let frame = 0;
this.blendTimer = window.setInterval(() => {    const t = Math.min(frame /
this.BLEND_FRAMES, 1);    incoming.setWeightForAllAnimatables(t);    if
(outgoing && outgoing !== incoming)
{    outgoing.setWeightForAllAnimatables(1 - t);    }    frame++;
if (frame > this.BLEND_FRAMES) {    clearInterval(this.blendTimer!);
this.blendTimer = null;    if (outgoing && outgoing !== incoming) {
outgoing.setWeightForAllAnimatables(0);    }    }    }, 1000 / 30);
}    // — Movement Update Loop

private
registerUpdateLoop(): void {    this.scene.onBeforeRenderObservable.add((
=> {    if (!this.rootMesh || !this.targetPosition || !this.isMoving)
return;    const delta = this.scene.getEngine().getDeltaTime() / 1000;

```

```

const current = this.rootMesh.position;          const dir =
this.targetPosition.subtract(current);          const dist = dir.length();
if (dist < ARRIVAL_THRESHOLD)
{
    this.rootMesh.position.copyFrom(this.targetPosition);
    this.isMoving = false;          this.targetPosition = null;
    this.crossFadeTo('idle');          return;          }          // Rotate toward
target          const angle = Math.atan2(dir.x, dir.z);
this.rootMesh.rotation.y = angle;          // Move toward target          const
step = dir.normalize().scale(WALK_SPEED * delta);
this.rootMesh.position.addInPlace(step);          });  } }

```

6. 3D Avatar Customizer UI

6.1 AvatarCustomizer.ts — The In-Game Customizer Screen

The avatar customizer is a full-screen HTML overlay rendered on top of the Babylon.js canvas. It is triggered when the player clicks their own avatar or presses the **Edit Avatar** button in the HUD. The left panel hosts a live 3D preview using the same avatar GLB in a lightweight preview scene. The right panel contains all the customization controls. The **AvatarData** interface is the canonical data shape serialized to and from the server.

```

// apps/client/src/ui/AvatarCustomizer.ts export interface AvatarData
{   bodyType: number;   // 0.0 (thin) to 1.0 (fat)   height: number;   //
0.0 (short) to 1.0 (tall)   build: number;   // 0.0 (slim) to 1.0
(muscular)   skinTone: string;   // hex e.g. '#F5CBA7'   hairColor:
string;   // hex   eyeColor: string;   // hex   topColor: string;   // hex
bottomColor: string; // hex } // Skin tone presets (12 swatches) const
SKIN_TONES = [   '#FDDDB4', '#F5CBA7', '#EAB88A', '#D4956A',
'#C07A50', '#A0522D', '#8B4513', '#6B3410',
'#4A2011', '#FDEDCD', '#F2D7C0', '#E8C4A0', ]; // Hair color presets (24
swatches) const HAIR_COLORS = [   '#1C1C1C', '#3B2314', '#5C3317', '#7B4B2A',
'#A0522D', '#C68642', '#D2A679', '#E8C49A',
'#F5DEB3', '#FAF0E6', '#B22222', '#8B0000',
'#FF4500', '#FF8C00', '#FFD700', '#808000',
'#2E8B57', '#006400', '#4169E1', '#00008B',
'#8B008B', '#4B0082', '#808080', '#FFFFFF', ]; // Eye color presets (8
swatches) const EYE_COLORS = [   '#4A3728', '#2D1B0E', '#5C7A29', '#2E8B57',
'#4169E1', '#1E3A8A', '#708090', '#C0C0C0', ]; export class AvatarCustomizer {
private overlay: HTMLElement | null = null; private previewController:
import('./AvatarController').AvatarController | null = null; private
currentData: AvatarData; private savedData: AvatarData; private
onSaveCallback: (data: AvatarData) => void; constructor(   savedData:
AvatarData,   onSave: (data: AvatarData) => void   ) {   this.savedData =
{ ...savedData };   this.currentData = { ...savedData };
this.onSaveCallback = onSave;   }   open(): void {   if (this.overlay)
return; // already open   this.overlay = this.buildOverlay();
document.body.appendChild(this.overlay);   }   close(): void
{   this.overlay?.remove();   this.overlay = null;
this.previewController?.dispose();   this.previewController =
null;   } // — UI Builder

```

```

private
buildOverlay(): HTMLElement {    const el = document.createElement('div');
el.id = 'avatar-customizer';    el.style.cssText = `                position: fixed;
inset: 0; background: rgba(0,0,0,0.85);                display: flex; align-items:
center; justify-content: center;                z-index: 9999; font-family: Calibri,
sans-serif;                `;    const panel = document.createElement('div');
panel.style.cssText = `                background: #1a1a2e; color: #e0e0e0; width:
820px; max-height: 90vh;                display: flex; gap: 0; overflow: hidden;
`;    panel.appendChild(this.buildPreviewPanel());
panel.appendChild(this.buildControlPanel());    el.appendChild(panel);
return el;    }    private buildPreviewPanel(): HTMLElement {    const div =
document.createElement('div');    div.style.cssText = 'width: 320px;
background: #0f0f23; position: relative; flex-shrink: 0;';    const canvas
= document.createElement('canvas');    canvas.id = 'avatar-preview-canvas';
canvas.style.cssText = 'width: 320px; height: 480px; display: block;';
div.appendChild(canvas);    // Initialize the preview Babylon scene
asynchronously after DOM insertion    setTimeout(() =>
this.initPreviewScene(canvas, 100);    const label =
document.createElement('p');    label.textContent = 'Live Preview';
label.style.cssText = 'text-align:center; color:#888; font-size:11pt;
margin:8px 0;';    div.appendChild(label);    return div;    }    private
async initPreviewScene(canvas: HTMLCanvasElement): Promise<void> {    const
BABYLON = await import('@babylonjs/core');    await
import('@babylonjs/loaders');    const { AvatarController } = await
import('../world/AvatarController');    const engine = new
BABYLON.Engine(canvas, true);    const scene = new BABYLON.Scene(engine);
scene.clearColor = new BABYLON.Color4(0.05, 0.05, 0.15, 1);    // Camera
orbiting the avatar    const cam = new BABYLON.ArcRotateCamera('previewCam',
-Math.PI / 2, Math.PI / 2.5, 3.5, new BABYLON.Vector3(0, 0.9, 0), scene);
cam.lowerRadiusLimit = 2;    cam.upperRadiusLimit = 5;
cam.attachControl(canvas, true);    new BABYLON.HemisphericLight('hemi',
new BABYLON.Vector3(0, 1, 0), scene);    this.previewController = new
AvatarController(scene, 'Preview');    await
this.previewController.load(BABYLON.Vector3.Zero());
this.previewController.applyCustomization(this.currentData);
engine.runRenderLoop(() => scene.render());
window.addEventListener('resize', () => engine.resize());    }    private
buildControlPanel(): HTMLElement {    const div =
document.createElement('div');    div.style.cssText = 'flex: 1; overflow-y:
auto; padding: 24px;';    div.innerHTML = '<h2 style="margin-top:0;
color:#7eb8f7;">Edit Avatar</h2>';    // Body Shape section
div.appendChild(this.buildSection('Body Shape',
[    this.buildSlider('Body Type', 'bodyType', 'Thin', 'Fat'),
this.buildSlider('Height', 'height', 'Short', 'Tall'),
this.buildSlider('Build', 'build', 'Slim', 'Muscular'),    ]));    //
Colors section    div.appendChild(this.buildSection('Colors',
[    this.buildSwatchPicker('Skin Tone', 'skinTone', SKIN_TONES, true),
this.buildSwatchPicker('Hair Color', 'hairColor', HAIR_COLORS, true),
this.buildSwatchPicker('Eye Color', 'eyeColor', EYE_COLORS, false),    ]));
// Clothing section    div.appendChild(this.buildSection('Clothing Colors',
[    this.buildColorInput('Top Color', 'topColor'),
this.buildColorInput('Bottom Color', 'bottomColor'),
this.buildNote('Full clothing items arrive in Phase 3.'),    ]));    //
Action buttons    const btnRow = document.createElement('div');
btnRow.style.cssText = 'margin-top: 20px; display: flex; gap: 12px;';
const saveBtn = this.buildButton('Save', '#2b5797', async () => {    await
this.persistToServer();    this.savedData = { ...this.currentData };
this.onSaveCallback(this.currentData);    this.close();    });    const
cancelBtn = this.buildButton('Cancel', '#555555', () =>
{    this.currentData = { ...this.savedData };    this.close();    });

```

```

btnRow.appendChild(saveBtn);      btnRow.appendChild(cancelBtn);
div.appendChild(btnRow);      return div;  }    // — Control Builders

private
buildSection(title: string, children: HTMLElement[]): HTMLElement {    const
sec = document.createElement('div');    sec.style.cssText = 'margin-bottom:
20px; border-bottom: 1px solid #333; padding-bottom: 16px;';    const h =
document.createElement('h3');    h.textContent = title;    h.style.cssText
= 'color:#aac4e8; font-size:12pt; margin-bottom:10px;';
sec.appendChild(h);    children.forEach(c => sec.appendChild(c));    return
sec;  }    private buildSlider(label: string, key: keyof AvatarData,
minLabel: string, maxLabel: string): HTMLElement {    const row =
document.createElement('div');    row.style.cssText = 'margin-bottom:
10px;';    const lbl = document.createElement('label');    lbl.textContent
= label;    lbl.style.cssText = 'display: block; font-size: 10.5pt; margin-
bottom: 4px;';    const controlRow = document.createElement('div');
controlRow.style.cssText = 'display: flex; align-items: center; gap: 8px;
font-size:9pt; color:#aaa;';    const minSpan =
document.createElement('span');    minSpan.textContent = minLabel;
const input = document.createElement('input');    input.type = 'range';
input.min = '0';    input.max = '100';    input.value =
String(Math.round((this.currentData[key] as number) * 100));
input.style.cssText = 'flex: 1; accent-color: #2b5797;';
input.addEventListener('input', () => {    (this.currentData[key] as
number)= parseInt(input.value) / 100;
this.previewController?.applyCustomization(this.currentData);    });
const maxSpan = document.createElement('span');    maxSpan.textContent =
maxLabel;    controlRow.appendChild(minSpan);
controlRow.appendChild(input);    controlRow.appendChild(maxSpan);
row.appendChild(lbl);    row.appendChild(controlRow);    return row;  }
private buildSwatchPicker(label: string, key: keyof AvatarData, swatches:
string[], showCustom: boolean): HTMLElement {    const div =
document.createElement('div');    div.style.cssText = 'margin-bottom:
12px;';    const lbl = document.createElement('label');    lbl.textContent
= label;    lbl.style.cssText = 'display:block; font-size:10.5pt; margin-
bottom:6px;';    div.appendChild(lbl);    const grid =
document.createElement('div');    grid.style.cssText = 'display:flex; flex-
wrap:wrap; gap:5px;';    swatches.forEach(color => {    const swatch =
document.createElement('button');    swatch.style.cssText = `
width:24px; height:24px; background:${color}; border:2px solid transparent;
cursor:pointer; padding:0;`;    swatch.title = color;    if
(this.currentData[key] === color) {    swatch.style.border = '2px solid
#ffffff';    }    swatch.addEventListener('click', () =>
{    (this.currentData[key] as string) = color;    // Reset borders
grid.querySelectorAll('button').forEach(b => { (b as
HTMLElement).style.border = '2px solid transparent'; });
swatch.style.border = '2px solid #ffffff';
this.previewController?.applyCustomization(this.currentData);    });
grid.appendChild(swatch);    });    if (showCustom) {    const
customInput = document.createElement('input');    customInput.type =
'color';    customInput.value = this.currentData[key] as string;
customInput.style.cssText = 'width:24px; height:24px; padding:0; border:none;
cursor:pointer;';    customInput.title = 'Custom color';
customInput.addEventListener('input', () => {    (this.currentData[key]
as string) = customInput.value;
this.previewController?.applyCustomization(this.currentData);    });
grid.appendChild(customInput);    }    div.appendChild(grid);    return
div;  }    private buildColorInput(label: string, key: keyof AvatarData):
HTMLElement {    const div = document.createElement('div');
div.style.cssText = 'margin-bottom: 10px; display:flex; align-items:center;
gap:10px;';    const lbl = document.createElement('label');

```

```

lbl.textContent = label;      lbl.style.cssText = 'font-size:10.5pt; flex:1;';
const input = document.createElement('input');      input.type = 'color';
input.value = this.currentData[key] as string;      input.style.cssText =
'width:36px; height:28px; padding:0; border:none; cursor:pointer;';
input.addEventListener('input', () => {      (this.currentData[key] as
string) = input.value;
this.previewController?.applyCustomization(this.currentData);      });
div.appendChild(lbl);      div.appendChild(input);      return div;  }
private buildNote(text: string): HTMLElement {      const p =
document.createElement('p');      p.textContent = text;      p.style.cssText =
'color:#666; font-size:9.5pt; font-style:italic; margin:4px 0 0 0;';
return p;  }      private buildButton(text: string, bg: string, onClick: () =>
void): HTMLElement {      const btn = document.createElement('button');
btn.textContent = text;      btn.style.cssText = `      background:${bg};
color:#fff; border:none; padding:8px 24px;      font-family:Calibri,sans-
serif; font-size:11pt; cursor:pointer;      `;
btn.addEventListener('click', onClick);      return btn;  }      // — Server
Persistence ————— private
async persistToServer(): Promise<void> {      const res = await
fetch('/api/users/me/avatar', {      method: 'PUT',      headers:
{ 'Content-Type': 'application/json' },      credentials: 'include',
body: JSON.stringify(this.currentData),      });      if (!res.ok)
{      throw new Error(`Failed to save avatar: ${res.status} $
{res.statusText}`);      }      } }

```

6.2 Server Route Update — Save Avatar Customization

Add the following route to `apps/server/src/routes/users.ts`. The Zod schema validates every field of `AvatarData` before writing to the database. After saving, the Socket.io event broadcasts the update to all players currently in the same room so their `RemoteAvatar` instances call `applyCustomization()` immediately.

```

// apps/server/src/routes/users.ts (additions) import { z } from 'zod';
import { prisma } from '../db'; import { io } from '../socket';      // your
Socket.io server instance import { SOCKET_EVENTS } from '@havenworld/shared';
const avatarDataSchema = z.object({      bodyType:      z.number().min(0).max(1),
height:      z.number().min(0).max(1),      build:
z.number().min(0).max(1),      skinTone:      z.string().regex(/^#[0-9A-Fa-f]{6}$/),
hairColor:      z.string().regex(/^#[0-9A-Fa-f]{6}$/),      eyeColor:
z.string().regex(/^#[0-9A-Fa-f]{6}$/),      topColor:      z.string().regex(/^#[0-
9A-Fa-f]{6}$/),      bottomColor: z.string().regex(/^#[0-9A-Fa-f]{6}$/), }); //
PUT /api/users/me/avatar router.put('/me/avatar', requireAuth, async (req,
res) => {      const parsed = avatarDataSchema.safeParse(req.body);      if (!
parsed.success) {      return res.status(400).json({ error: 'Invalid avatar
data', issues: parsed.error.issues });      }      const userId = req.user!.id; //
injected by requireAuth middleware      const updatedUser = await
prisma.user.update({      where: { id: userId },      data:
{ avatarCustomization: parsed.data },      });      // Broadcast to the room the
user is currently in      const userSession = await
prisma.userSession.findFirst({      where: { userId, isActive: true },
select: { currentRoomId: true },      });      if (userSession?.currentRoomId) {
io.to(`room:${
userSession.currentRoomId}`).emit(SOCKET_EVENTS.AVATAR_UPDATE,
{ userId, avatarData: parsed.data });      }      return res.json({ success:
true, avatarCustomization: updatedUser.avatarCustomization }); });

```

7. Furniture System

7.1 Furniture Data Model Review

The Prisma schema established in Part 4A already contains the three tables needed for the furniture system. In Phase 2 these tables are used as-is; no new migrations are needed for the core furniture feature. The `assetUrl` column on the `items` table stores the path to the furniture's GLB file. At runtime, the `FurnitureManager` constructs the full URL as `/assets/furniture/{itemId}.glb` but the column stores the canonical relative path for portability.

Table	Relevant Columns	Usage in Phase 2
<code>items</code>	<code>id</code> , <code>name</code> , <code>type</code> , <code>assetUrl</code> , <code>price</code> , <code>isDefault</code>	Catalog of all available furniture GLBs. <code>type = 'furniture'</code> filters to furniture items.
<code>inventories</code>	<code>userId</code> , <code>itemId</code> , <code>quantity</code>	Tracks which items the player owns. <code>Quantity ≥ 1</code> means the player can place that furniture.
<code>room_furniture</code>	<code>id</code> , <code>roomId</code> , <code>itemId</code> , <code>placedById</code> , <code>x</code> , <code>y</code> , <code>z</code> , <code>rotY</code> , <code>scaleX</code> , <code>scaleY</code> , <code>scaleZ</code>	Current layout of placed furniture in each room. Loaded on room entry; saved on layout commit.

7.2 FurnitureManager.ts — Loading and Placing Furniture

```
// apps/client/src/world/FurnitureManager.ts import * as BABYLON from '@babylonjs/core'; import '@babylonjs/loaders'; export interface PlacedFurniture { id: string; itemId: string; mesh: BABYLON.AbstractMesh; isOwned: boolean; // true if current user placed this piece } interface FurniturePlacementData { id: string; itemId: string; assetUrl: string; placedById: string; x: number; y: number; z: number; rotY: number; scaleX: number; scaleY: number; scaleZ: number; } export class FurnitureManager { private scene: BABYLON.Scene; private currentUserId: string; private placed: Map<string, PlacedFurniture> = new Map(); // GLB template cache: itemId -> root mesh template // Prevents loading the same GLB multiple times private templateCache: Map<string, BABYLON.AbstractMesh> = new Map(); constructor(scene: BABYLON.Scene, currentUserId: string) { this.scene = scene; this.currentUserId = currentUserId; } // — Load Room Furniture async loadRoomFurniture(roomId: string): Promise<void> { const res = await fetch(`~/api/rooms/${roomId}/furniture`, { credentials: 'include', }); if (!res.ok) { console.error(`[FurnitureManager] Failed to fetch furniture for room ${roomId}`); return; }
```



```

const items: FurniturePlacementData[] = await res.json();      await
Promise.all(items.map(item => this.placeItem(item)));
console.log(`[FurnitureManager] Loaded ${items.length} furniture
items.`);    }    // — Place a Single Item

FurniturePlacementData): Promise<PlacedFurniture | null> {      async placeItem(data:
BABYLON.AbstractMesh;      if (this.templateCache.has(data.itemId))
{      // Re-use cached template via instancing (one draw call per mesh)
const template = this.templateCache.get(data.itemId)!;      instance =
(template as BABYLON.Mesh).createInstance(`${data.itemId}_${data.id}`) as
BABYLON.AbstractMesh;      } else {      // First load: import the GLB and
cache the root mesh as template      const assetPath =
data.assetUrl.startsWith('/') ? data.assetUrl :
`/assets/furniture/${data.itemId}.glb`;      const pathParts =
assetPath.split('/');      const fileName = pathParts.pop()!;      const
folder = pathParts.join('/') + '/';      const result = await
BABYLON.SceneLoader.ImportMeshAsync('', folder, fileName, this.scene);
const root = result.meshes[0];      root.setEnabled(false); // hide template
root.name = `template_${data.itemId}`;
this.templateCache.set(data.itemId, root);      instance = (root as
BABYLON.Mesh).createInstance(`${data.itemId}_${data.id}`) as
BABYLON.AbstractMesh;      instance.setEnabled(true);      }
instance.position = new BABYLON.Vector3(data.x, data.y, data.z);
instance.rotation = new BABYLON.Vector3(0, data.rotY, 0);
instance.scaling = new BABYLON.Vector3(data.scaleX, data.scaleY,
data.scaleZ);      instance.isPickable = true;      const placedItem:
PlacedFurniture = {      id: data.id,      itemId: data.itemId,      mesh:
instance,      isOwned: data.placedById === this.currentUserId,      };
this.placed.set(data.id, placedItem);      return placedItem;    }    // —
Picking

pickFurniture(pickInfo: BABYLON.PickingInfo): PlacedFurniture | null {      if
(!pickInfo.hit || !pickInfo.pickedMesh) return null;      for (const [, pf] of
this.placed) {      if (pf.mesh === pickInfo.pickedMesh) return pf;      }
return null;    }    // — Remove Item

removeItem(id:
string): void {      const item = this.placed.get(id);      if (item)
{      item.mesh.dispose();      this.placed.delete(id);      }    }    //
— Clear Room

clear(): void {      this.placed.forEach(pf => pf.mesh.dispose());
this.placed.clear();      this.templateCache.forEach(t => t.dispose());
this.templateCache.clear();    }    get allPlaced(): Map<string,
PlacedFurniture> {      return this.placed;    }

```

7.3 Creating Placeholder Furniture GLBs

Real furniture art takes time to produce. The following Blender Python script generates simple geometric placeholder GLBs for all six default furniture items. Run it in Blender's **Scripting** workspace, then copy the outputs to `public/assets/furniture/`.

```

## Run in Blender 5.1 Scripting workspace ## Generates placeholder GLBs:
chair, table, sofa, lamp, rug, bookshelf import bpy, os, math OUTPUT_DIR =
"/tmp/havenworld_furniture/" os.makedirs(OUTPUT_DIR, exist_ok=True) def
add_material(obj, name, r, g, b):      mat = bpy.data.materials.new(name=name)
mat.use_nodes = True      bsdf = mat.node_tree.nodes.get('Principled BSDF')
if bsdf:      bsdf.inputs['Base Color'].default_value = (r, g, b, 1.0)
bsdf.inputs['Roughness'].default_value = 0.7

```

```

obj.data.materials.append(mat) FURNITURE = {      "chair": {      "parts":
[      {"type":"cube", "name":"seat", "loc":(0,0,0.45), "scale":
(0.5,0.5,0.05)},      {"type":"cube", "name":"back", "loc":(0, -
0.225,0.75), "scale":(0.5,0.05,0.3)},
{"type":"cube", "name":"leg_fl", "loc":(-0.2,0.2,0.22), "scale":
(0.05,0.05,0.44)},      {"type":"cube", "name":"leg_fr", "loc":
(0.2,0.2,0.22), "scale":(0.05,0.05,0.44)},
{"type":"cube", "name":"leg_bl", "loc":(-0.2, -0.2,0.22), "scale":
(0.05,0.05,0.44)},      {"type":"cube", "name":"leg_br", "loc":(0.2, -
0.2,0.22), "scale":(0.05,0.05,0.44)},      ],      "color": (0.55, 0.27,
0.07),      },      "table": {      "parts":
[      {"type":"cube", "name":"top", "loc":(0,0,0.75), "scale":
(1.0,0.6,0.04)},      {"type":"cube", "name":"leg_fl", "loc":(-
0.45,0.25,0.37), "scale":(0.06,0.06,0.74)},
{"type":"cube", "name":"leg_fr", "loc":(0.45,0.25,0.37), "scale":
(0.06,0.06,0.74)},      {"type":"cube", "name":"leg_bl", "loc":(-0.45, -
0.25,0.37), "scale":(0.06,0.06,0.74)},
{"type":"cube", "name":"leg_br", "loc":(0.45, -0.25,0.37), "scale":
(0.06,0.06,0.74)},      ],      "color": (0.6, 0.4, 0.2),      },
"sofa": {      "parts": [      {"type":"cube", "name":"base", "loc":
(0,0,0.25), "scale":(1.8,0.8,0.3)},
{"type":"cube", "name":"back", "loc":(0, -0.35,0.55), "scale":(1.8,0.1,0.3)},
{"type":"cube", "name":"arm_l", "loc":(-0.85,0,0.45), "scale":(0.1,0.8,0.4)},
{"type":"cube", "name":"arm_r", "loc":(0.85,0,0.45), "scale":(0.1,0.8,0.4)},
],      "color": (0.2, 0.4, 0.8),      },      "lamp": {      "parts": [
{"type":"cylinder", "name":"pole", "loc":(0,0,0.75), "scale":(0.04,0.04,1.5)},
{"type":"cylinder", "name":"shade", "loc":(0,0,1.55), "scale":(0.25,0.25,0.35)},
{"type":"cylinder", "name":"base", "loc":(0,0,0.04), "scale":(0.18,0.18,0.08)},
],      "color": (0.85, 0.85, 0.75),      },      "rug": {      "parts":
[      {"type":"cube", "name":"rug", "loc":(0,0,0.01), "scale":
(2.0,1.4,0.02)},      ],      "color": (0.7, 0.15, 0.15),      },
"bookshelf": {      "parts":
[      {"type":"cube", "name":"frame", "loc":(0,0,0.9), "scale":
(0.9,0.3,1.8)},      {"type":"cube", "name":"shelf1", "loc":
(0,0,0.35), "scale":(0.88,0.28,0.04)},
{"type":"cube", "name":"shelf2", "loc":(0,0,0.85), "scale":(0.88,0.28,0.04)},
{"type":"cube", "name":"shelf3", "loc":(0,0,1.35), "scale":(0.88,0.28,0.04)},
],      "color": (0.45, 0.22, 0.05),      }, } for item_name, config in
FURNITURE.items():      # Clear scene
bpy.ops.object.select_all(action='SELECT')      bpy.ops.object.delete()
created = []      for part in config["parts"]:      if part["type"] ==
"cube":      bpy.ops.mesh.primitive_cube_add(size=1,
location=part["loc"])      elif part["type"] == "cylinder":
bpy.ops.mesh.primitive_cylinder_add(radius=0.5, depth=1,
location=part["loc"])      obj = bpy.context.active_object
obj.name = f"{item_name}_{part['name']}"      obj.scale = part["scale"]
bpy.ops.object.transform_apply(scale=True)      r, g, b = config["color"]
add_material(obj, f"mat_{item_name}", r, g, b)      created.append(obj)
# UV unwrap all parts      bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.editmode_toggle()      bpy.ops.mesh.select_all(action='SELECT')
bpy.ops.uv.smart_project()      bpy.ops.object.editmode_toggle()      # Export
GLB (all objects in scene)      filepath = os.path.join(OUTPUT_DIR,
f"{item_name}.glb")      bpy.ops.export_scene.gltf(      filepath=filepath,
export_format='GLB',      use_selection=False,      export_apply=True,
export_materials='EXPORT',      )      print(f"[OK] Exported: {filepath}")
print("All furniture GLBs exported to", OUTPUT_DIR)

```

After running the script, copy all six GLB files from /tmp/havenworld_furniture/ to public/assets/furniture/:

```
# From your HavenWorld project root: cp /tmp/havenworld_furniture/*.glb
public/assets/furniture/ # Optimize all furniture GLBs for f in
public/assets/furniture/*.glb; do npx @gltf-transform/cli optimize "$f"
"$f" --texture-compress webp echo "Optimized: $f" done
```

8. Personal Room Editor

8.1 Overview of the Room Editor

The room editor activates when a player enters their own personal room. The HUD detects room ownership by comparing `room.ownerId` against the authenticated user's ID, which is available from the session context established in Part 4A. A **Decorate** button appears in the top-right corner of the HUD only under this condition. Clicking it calls `RoomEditor.enterEditMode()`.

In Edit Mode, the scene transitions to an editing state with these visual and interactive changes: a translucent floor grid snaps furniture placement to 0.5-meter increments; a left-panel inventory drawer slides in listing the player's owned furniture items with click-to-place; a ghost mesh (translucent, blue-tinted clone of the selected furniture) tracks the mouse cursor on the floor plane; left-clicking places the ghost; right-clicking an already-placed item opens a context menu with Move, Rotate 90°, and Remove options. A top bar shows Save Layout and Cancel buttons.

8.2 RoomEditor.ts — Complete Implementation

```
// apps/client/src/world/RoomEditor.ts import * as BABYLON from
'@babylonjs/core'; import { GridMaterial } from '@babylonjs/materials';
import type { FurnitureManager, PlacedFurniture } from './FurnitureManager';
export interface FurniturePlacement { id: string; // UUID (temp for
new items, DB id for existing) itemId: string; assetUrl: string; x:
number; y: number; z: number; rotY: number; scaleX: number; scaleY:
number; scaleZ: number; isNew: boolean; // true if not yet saved to
server isRemoved: boolean; // true if user removed it in this session }
export class RoomEditor { private scene: BABYLON.Scene; private
furnitureManager: FurnitureManager; private roomId: string; private
floorY = 0; private isEditing = false; private ghostMesh:
BABYLON.AbstractMesh | null = null; private ghostItemId: string | null =
null; private ghostAssetUrl: string | null = null; private gridMesh:
BABYLON.Mesh | null = null; private selectedFurniture: PlacedFurniture |
null = null; private pointerObserver: BABYLON.Observer<BABYLON.PointerInfo>
| null = null; private contextMenu: HTMLElement | null = null; // Tracks
all changes made in this edit session private pendingChanges: Map<string,
FurniturePlacement> = new Map(); constructor(scene: BABYLON.Scene,
furnitureManager: FurnitureManager, roomId: string, floorY = 0)
{ this.scene = scene; this.furnitureManager = furnitureManager;
```

```

this.roomId = roomId;      this.floorY = floorY;  }    // — Enter / Exit
Edit Mode ————— enterEditMode():
void {    if (this.isEditing) return;      this.isEditing = true;
this.pendingChanges.clear();    this.showGrid();
this.attachPointerObserver();    this.buildInventoryPanel();
console.log('[RoomEditor] Edit mode ON');  }    exitEditMode(): void
{    if (!this.isEditing) return;      this.isEditing = false;
this.hideGrid();    this.clearGhostMesh();    this.detachPointerObserver();
this.removeInventoryPanel();    this.removeContextMenu();
this.selectedFurniture = null;    console.log('[RoomEditor] Edit mode OFF');
}    // — Grid Helper

private
showGrid(): void {    this.gridMesh = BABYLON.MeshBuilder.CreateGround(
'editor_grid',    { width: 20, height: 20, subdivisions: 40 },
this.scene    );    this.gridMesh.position.y = this.floorY + 0.002; //
slightly above floor    this.gridMesh.isPickable = false;    const gridMat
= new GridMaterial('editor_grid_mat', this.scene);
gridMat.majorUnitFrequency = 5;    gridMat.minorUnitVisibility = 0.45;
gridMat.gridRatio = 0.5;    // 0.5m grid lines
gridMat.backFaceCulling = false;    gridMat.mainColor = new
BABYLON.Color3(0.2, 0.6, 1.0);    gridMat.lineColor = new
BABYLON.Color3(0.4, 0.7, 1.0);    gridMat.opacity = 0.5;
this.gridMesh.material = gridMat;  }    private hideGrid(): void
{    this.gridMesh?.dispose();    this.gridMesh = null;  }    // — Ghost
Mesh (Placement Preview) ————— async
startPlacement(itemId: string, assetUrl: string): Promise<void>
{    this.clearGhostMesh();    this.ghostItemId = itemId;
this.ghostAssetUrl = assetUrl;    const pathParts = assetUrl.split('/');
const fileName = pathParts.pop()!;    const folder = pathParts.join('/') +
'/';    const result = await BABYLON.SceneLoader.ImportMeshAsync('',
folder, fileName, this.scene);    this.ghostMesh = result.meshes[0];
this.ghostMesh.name = 'ghost_placement';    this.ghostMesh.isPickable =
false;    // Apply translucent blue ghost material to all sub-meshes
result.meshes.forEach(mesh => {    if (mesh.material) {    const
ghostMat = new BABYLON.StandardMaterial(`ghost_mat_${mesh.name}`,
this.scene);    ghostMat.diffuseColor = new BABYLON.Color3(0.3, 0.5,
1.0);    ghostMat.alpha = 0.5;    ghostMat.backFaceCulling = false;
mesh.material = ghostMat;    mesh.isPickable =
false;    }    });    private clearGhostMesh(): void
{    this.ghostMesh?.dispose();    this.ghostMesh = null;
this.ghostItemId = null;    this.ghostAssetUrl = null;  }    // — Pointer
Handling ————— private
attachPointerObserver(): void {    this.pointerObserver =
this.scene.onPointerObservable.add((pointerInfo) => {    switch
(pointerInfo.type) {    case BABYLON.PointerEventTypes.POINTERMOVE:
this.onPointerMove(pointerInfo);    break;    case
BABYLON.PointerEventTypes.POINTERDOWN:    if (pointerInfo.event.button
=== 0) this.onPointerDown(pointerInfo);    if
(pointerInfo.event.button === 2) this.onRightClick(pointerInfo);
break;    }    });    private detachPointerObserver(): void {    if
(this.pointerObserver)
{    this.scene.onPointerObservable.remove(this.pointerObserver);
this.pointerObserver = null;    }    }    private onPointerMove(evt:
BABYLON.PointerInfo): void {    if (!this.ghostMesh) return;    const pick
= this.scene.pick(    this.scene.pointerX,    this.scene.pointerY,
mesh => mesh.name.startsWith('walkable_') || mesh.name ===
'editor_grid'    );    if (pick?.hit && pick.pickedPoint) {    const
snapped = this.snapToGrid(pick.pickedPoint);    this.ghostMesh.position =
snapped;    }    }    private onPointerDown(evt: BABYLON.PointerInfo): void {
this.removeContextMenu();    if (this.ghostMesh && this.ghostItemId &&

```

```

this.ghostAssetUrl) { // Place the ghost mesh as a real furniture item
this.placeFurnitureFromGhost(); return; } // Try to select
existing furniture const pick = this.scene.pick(this.scene.pointerX,
this.scene.pointerY); const hit =
this.furnitureManager.pickFurniture(pick!); if (hit)
{ this.selectedFurniture = hit; } else
{ this.selectedFurniture = null; } } private onRightClick(evt:
BABYLON.PointerInfo): void { const pick =
this.scene.pick(this.scene.pointerX, this.scene.pointerY); const hit =
this.furnitureManager.pickFurniture(pick!); if (hit)
{ this.selectedFurniture = hit; this.showContextMenu(hit,
evt.event as PointerEvent); } } // — Furniture Placement

private async
placeFurnitureFromGhost(): Promise<void> { if (!this.ghostMesh || !
this.ghostItemId || !this.ghostAssetUrl) return; const pos =
this.ghostMesh.position.clone(); const tempId = `new_${
crypto.randomUUID()}`; const placement: FurniturePlacement =
{ id: tempId, itemId: this.ghostItemId, assetUrl:
this.ghostAssetUrl, x: pos.x, y: pos.y, z: pos.z, rotY: 0,
scaleX: 1, scaleY: 1, scaleZ: 1, isNew: true, isRemoved: false,
}; this.pendingChanges.set(tempId, placement); await
this.furnitureManager.placeItem({ id: tempId, itemId:
placement.itemId, assetUrl: placement.assetUrl, placedById:
'local', x: placement.x, y: placement.y, z: placement.z, rotY:
placement.rotY, scaleX: placement.scaleX, scaleY:
placement.scaleY, scaleZ: placement.scaleZ, });
this.clearGhostMesh(); } rotateFurniture(id: string, deltaRotY: number):
void { const item = this.furnitureManager.allPlaced.get(id); if (!
item) return; item.mesh.rotation.y += deltaRotY; if
(this.pendingChanges.has(id)) { this.pendingChanges.get(id)!.rotY +=
deltaRotY; } else { // Create a change record for an existing item
this.pendingChanges.set(id, { id, itemId: item.itemId,
assetUrl: `/assets/furniture/${item.itemId}.glb`, x:
item.mesh.position.x, y: item.mesh.position.y, z:
item.mesh.position.z, rotY: item.mesh.rotation.y, scaleX:
item.mesh.scaling.x, scaleY: item.mesh.scaling.y, scaleZ:
item.mesh.scaling.z, isNew: false, isRemoved:
false, }); } removeFurniture(id: string): void
{ this.furnitureManager.removeItem(id); if
(this.pendingChanges.has(id)) { this.pendingChanges.get(id)!.isRemoved
= true; } else { this.pendingChanges.set(id, { id, itemId:
'', assetUrl: '', x: 0, y: 0, z: 0, rotY: 0, scaleX: 1,
scaleY: 1, scaleZ: 1, isNew: false, isRemoved: true, }); } }
this.removeContextMenu(); this.selectedFurniture = null; } // —
Grid Snapping

private snapToGrid(pos: BABYLON.Vector3): BABYLON.Vector3 { const snap =
0.5; const snappedX = Math.round(pos.x / snap) * snap; const snappedZ
= Math.round(pos.z / snap) * snap; // Y: floor height + half bounding box
height of ghost mesh let snappedY = this.floorY; if (this.ghostMesh)
{ const bb = this.ghostMesh.getBoundingInfo().boundingBox;
snappedY = this.floorY + bb.extendSize.y; } return new
BABYLON.Vector3(snappedX, snappedY, snappedZ); } // — Context Menu

private
showContextMenu(item: PlacedFurniture, evt: PointerEvent): void
{ this.removeContextMenu(); const menu =
document.createElement('div'); menu.id = 'furniture-context-menu';
menu.style.cssText = `position: fixed; left: ${evt.clientX}px; top: $
{evt.clientY}px; background: #1a1a2e; border: 1px solid #2b5797; color:
#e0e0e0; z-index: 9998; font-family: Calibri, sans-serif; font-size:
11pt; min-width: 140px;`; const options = [ { label:

```

```

'Move', action: () => this.startPlacement(item.itemId, `/assets/furniture/${
item.itemId}.glb`), { label: 'Rotate 90°', action: () =>
this.rotateFurniture(item.id, Math.PI / 2) }, { label: 'Remove',
action: () => this.removeFurniture(item.id) }, ];
options.forEach(opt => { const btn = document.createElement('button');
btn.textContent = opt.label; btn.style.cssText = `
display:block; width:100%; background:none; border:none; color:#e0e0e0;
padding:8px 16px; text-align:left; cursor:pointer; font-size:11pt;
font-family:Calibri,sans-serif;`;
btn.addEventListener('mouseenter', () => { btn.style.background =
'#2b5797'; }); btn.addEventListener('mouseleave', () =>
{ btn.style.background = 'none'; }); btn.addEventListener('click', ()
=> { opt.action(); this.removeContextMenu(); }); menu.appendChild(btn);
}); document.body.appendChild(menu); this.contextMenu =
menu; // Close on outside click setTimeout(() =>
{ document.addEventListener('click', () => this.removeContextMenu(),
{ once: true }); }, 0); } private removeContextMenu(): void
{ this.contextMenu?.remove(); this.contextMenu = null; } // —
Inventory Panel
private buildInventoryPanel(): void { const panel =
document.createElement('div'); panel.id = 'room-editor-inventory';
panel.style.cssText = `position:fixed; left:0; top:50%;
transform:translateY(-50%); background:#1a1a2e; border-right:2px solid
#2b5797; width:180px; max-height:70vh; overflow-y:auto; z-index:9000;
font-family:Calibri,sans-serif; color:#e0e0e0; padding:12px;`;
panel.innerHTML = '<h4 style="margin:0 0 10px; color:#7eb8f7; font-
size:11pt;">Inventory</h4>'; // Fetch inventory and populate
fetch('/api/users/me/inventory?type=furniture', { credentials: 'include' })
.then(r => r.json()) .then((items: Array<{itemId:string; name:string;
assetUrl:string; quantity:number}>)) => { if (!items.length) {
panel.innerHTML += '<p style="color:#888; font-size:9.5pt;">No furniture
owned.</p>'; return; } items.forEach(item => {
const btn = document.createElement('button'); btn.textContent =
item.name; btn.title = `Place ${item.name} (x${item.quantity})`;
btn.style.cssText = `display:block; width:100%;
background:#0f0f23; border:1px solid #333; color:#e0e0e0;
padding:6px 8px; margin-bottom:6px; text-align:left;
cursor:pointer; font-size:10pt; font-family:Calibri,sans-serif;`;
btn.addEventListener('click', () => this.startPlacement(item.itemId,
item.assetUrl)); panel.appendChild(btn); });
// Save / Cancel buttons const footer = document.createElement('div');
footer.style.cssText = 'margin-top:16px; border-top:1px solid #333; padding-
top:10px;'; const saveBtn = document.createElement('button');
saveBtn.textContent = 'Save Layout'; saveBtn.style.cssText = `width:100%;
background:#2b5797; color:#fff; border:none; padding:7px;
cursor:pointer; font-size:10.5pt; font-family:Calibri,sans-serif; margin-
bottom:6px;`; saveBtn.addEventListener('click', () => this.saveLayout());
const cancelBtn = document.createElement('button'); cancelBtn.textContent
= 'Cancel'; cancelBtn.style.cssText = `width:100%; background:#555;
color:#fff; border:none; padding:7px; cursor:pointer; font-size:10.5pt;
font-family:Calibri,sans-serif;`; cancelBtn.addEventListener('click', ()
=> this.cancelEdit()); footer.appendChild(saveBtn);
footer.appendChild(cancelBtn); panel.appendChild(footer);
document.body.appendChild(panel); } private removeInventoryPanel(): void
{ document.getElementById('room-editor-inventory')?.remove(); } //
— Save / Cancel —
async saveLayout(): Promise<void> { const layout =
Array.from(this.pendingChanges.values()).filter(p => !p.isRemoved);
const res = await fetch(`/api/rooms/${this.roomId}/furniture/layout`, {
method: 'POST', headers: { 'Content-Type': 'application/json' },

```

```

credentials: 'include',      body: JSON.stringify({ layout }),    });
if (!res.ok) {      alert(`Failed to save layout: ${res.status}. Please try
again.`);      return;    }      this.pendingChanges.clear();
this.exitEditMode();      console.log('[RoomEditor] Layout saved
successfully.');
```

```

    }      cancelEdit(): void {      // Reload the room furniture
from the server (discard all pending changes)
this.furnitureManager.clear();
this.furnitureManager.loadRoomFurniture(this.roomId);
this.pendingChanges.clear();      this.exitEditMode();    } }

```

8.3 Server Route — Save Room Layout

Add the following endpoint to `apps/server/src/routes/rooms.ts`. The route validates room ownership, atomically replaces the entire layout in a Prisma transaction, and broadcasts the update to all connected players in the room:

```

// apps/server/src/routes/rooms.ts (additions) import { z } from 'zod';
import { prisma } from '../db'; import { io } from '../socket'; import
{ SOCKET_EVENTS } from '@havenworld/shared'; const furniturePlacementSchema
= z.object({ id: z.string(), itemId: z.string(), x:
z.number(), y: z.number(), z: z.number(), rotY:
z.number(), scaleX: z.number().default(1), scaleY:
z.number().default(1), scaleZ: z.number().default(1), isNew:
z.boolean(), isRemoved: z.boolean(), }); const layoutBodySchema =
z.object({ layout: z.array(furniturePlacementSchema), }); // POST
/api/rooms/:roomId/furniture/layout router.post('/:roomId/furniture/layout',
requireAuth, async (req, res) => { const { roomId } = req.params; const
userId = req.user!.id; // Verify room ownership const room = await
prisma.room.findUnique({ where: { id: roomId } }); if (!room) return
res.status(404).json({ error: 'Room not found' }); if (room.ownerId !==
userId) { return res.status(403).json({ error: 'You do not own this room'
}); } const parsed = layoutBodySchema.safeParse(req.body); if (!
parsed.success) { return res.status(400).json({ error: 'Invalid layout
data', issues: parsed.error.issues }); } const { layout } = parsed.data;
const activeItems = layout.filter(p => !p.isRemoved); // Atomic
transaction: delete all existing, insert new await prisma.$transaction([
prisma.roomFurniture.deleteMany({ where: { roomId } }),
prisma.roomFurniture.createMany({ data: activeItems.map(p => ({
roomId, itemId: p.itemId, placedById: userId, x: p.x,
y: p.y, z: p.z, rotY: p.rotY, scaleX: p.scaleX,
scaleY: p.scaleY, scaleZ: p.scaleZ, )), )), )), //
Real-time sync: broadcast to all players in the room io.to(`room:$
{roomId}`).emit(SOCKET_EVENTS.ROOM_FURNITURE_UPDATED, { roomId }); return
res.json({ success: true, placedCount: activeItems.length }); });

```

8.4 Socket Events for Real-Time Furniture Sync

Add the following constants to `packages/shared/src/index.ts` so both the client and server reference the same event strings without typos:

```

// packages/shared/src/index.ts (additions to SOCKET_EVENTS) export const
SOCKET_EVENTS = { // ... all existing events from Part 4A ... // Avatar
AVATAR_UPDATE: 'avatar:update', // Furniture real-time sync
ROOM_FURNITURE_UPDATED: 'room:furniture_updated', FURNITURE_PLACE:
'furniture:place', FURNITURE_REMOVE: 'furniture:remove', } as

```

```
const; export type SocketEventKey = keyof typeof SOCKET_EVENTS; export type
SocketEventValue = (typeof SOCKET_EVENTS)[SocketEventKey];
```

In the server's Socket.io connection handler (apps/server/src/socket.ts), add handlers for the per-item events:

```
// apps/server/src/socket.ts (additions inside io.on('connection', ...))
socket.on(SOCKET_EVENTS.FURNITURE_PLACE, async (data) => { const { roomId,
placement } = data; const userId = socket.data.userId; // set by auth
middleware const room = await prisma.room.findUnique({ where: { id: roomId
} }); if (!room || room.ownerId !== userId) { return
socket.emit('error', { message: 'Not authorized to place furniture in this
room' }); } const created = await
prisma.roomFurniture.create({ data: { roomId, itemId:
placement.itemId, placedById: userId, x: placement.x, y: placement.y,
z: placement.z, rotY: placement.rotY, scaleX: 1, scaleY: 1, scaleZ: 1,
}, }); io.to(`room:${
roomId}`).emit(SOCKET_EVENTS.ROOM_FURNITURE_UPDATED, { roomId, action:
'place', item: created }); }); socket.on(SOCKET_EVENTS.FURNITURE_REMOVE,
async (data) => { const { roomId, furnitureId } = data; const userId =
socket.data.userId; const item = await
prisma.roomFurniture.findUnique({ where: { id: furnitureId } }); if (!item
|| item.placedById !== userId) { return socket.emit('error', { message:
'Not authorized to remove this item' }); } await
prisma.roomFurniture.delete({ where: { id: furnitureId } }); io.to(`room:${
roomId}`).emit(SOCKET_EVENTS.ROOM_FURNITURE_UPDATED, { roomId, action:
'remove', furnitureId }); });
```

9. Seed Data: Default Furniture Items

Update prisma/seed.ts to insert the ten default furniture items. The first six are `isDefault: true` and are granted to every new user automatically at registration (the registration handler in Part 4A already runs inventory seeding). The last four are purchasable items available in the shop (Phase 3 feature) but registered in the database now so the asset pipeline can be built and tested.

```
// prisma/seed.ts (append after existing seed code) const furnitureItems = [
{ name: 'Wooden Chair', type: 'furniture', description: 'A simple
wooden chair. Comfortable enough.', assetUrl:
'/assets/furniture/chair.glb', price: 0, isDefault: true, }, {
name: 'Round Table', type: 'furniture', description: 'A round wooden
table. Fits four chairs.', assetUrl: '/assets/furniture/table.glb',
price: 0, isDefault: true, }, { name: 'Blue Sofa', type:
'furniture', description: 'A comfortable two-seater sofa in ocean blue.',
assetUrl: '/assets/furniture/sofa.glb', price: 0, isDefault:
true, }, { name: 'Floor Lamp', type: 'furniture',
description: 'A tall floor lamp with a warm white shade.', assetUrl:
'/assets/furniture/lamp.glb', price: 0, isDefault: true, }, {
name: 'Bookshelf', type: 'furniture', description: 'A wooden
bookshelf. Perfect for the intellectual look.', assetUrl:
'/assets/furniture/bookshelf.glb', price: 0, isDefault: true, },
{ name: 'Red Rug', type: 'furniture', description: 'A classic red
area rug. Ties the room together.', assetUrl:
```



```

'/assets/furniture/rug.glb',    price: 0,    isDefault: true,    },
{    name: 'Wall Painting',    type: 'furniture',    description: 'An
abstract painting to brighten up your walls.',    assetUrl:
'/assets/furniture/painting.glb',    price: 100,    isDefault: false,    },
{    name: 'Fireplace',    type: 'furniture',    description: 'A cozy
decorative fireplace. Crackling not included.',    assetUrl:
'/assets/furniture/fireplace.glb',    price: 250,    isDefault: false,    },
{    name: 'Plant Pot',    type: 'furniture',    description: 'A leafy
green houseplant in a terracotta pot.',    assetUrl:
'/assets/furniture/plant.glb',    price: 50,    isDefault: false,    }, {
name: 'TV Stand',    type: 'furniture',    description: 'A sleek TV stand.
Screen sold separately.',    assetUrl: '/assets/furniture/tv.glb',
price: 200,    isDefault: false,    }, ]; for (const item of furnitureItems)
{    await prisma.item.upsert({    where: { name: item.name },    create:
item,    update: {},    }); } // Grant all isDefault furniture to all
existing users const defaultItems = await prisma.item.findMany({    where:
{ type: 'furniture', isDefault: true }, }); const allUsers = await
prisma.user.findMany({ select: { id: true } }); for (const user of allUsers)
{    for (const item of defaultItems) {    await prisma.inventory.upsert({
where: { userId_itemId: { userId: user.id, itemId: item.id } },    create:
{ userId: user.id, itemId: item.id, quantity: 1 },    update: {},    });
} } console.log(`Seeded ${furnitureItems.length} furniture items.`);

```

Re-run the seed after adding:

```
pnpm --filter server prisma db seed
```

10. Performance Optimization for 3D Browser Games

10.1 Asset Loading Strategy

The most impactful performance decision in a furniture-heavy social game is **mesh instancing**. When ten players each have a wooden chair in their personal room, a naive implementation would load `chair.glb` ten times, creating ten separate GPU draw calls per frame. Babylon.js solves this with `InstancedMesh`: the geometry and material data are uploaded to the GPU once from a template mesh, and each additional placement is rendered by submitting only a transform matrix per instance. This reduces GPU overhead from $O(N)$ draw calls to $O(1)$ for geometry submission plus $O(N)$ for transforms — a dramatic improvement for scenes with many repeated objects.

The `FurnitureManager` in Section 7.2 already implements this pattern via `templateCache` and `createInstance()`. As a reminder, here is the core pattern isolated for clarity:

```
// First placement of 'chair' – loads GLB once const result = await
BABYLON.SceneLoader.ImportMeshAsync(  '', '/assets/furniture/', 'chair.glb',
scene ); const template = result.meshes[0] as BABYLON.Mesh;
template.setEnabled(false); // hide the template, never render it directly
templateCache.set('chair', template); // Every subsequent chair – zero
additional GPU memory, one matrix per instance const instance =
template.createInstance(`chair_${uniqueId}`); instance.position = new
BABYLON.Vector3(x, y, z); instance.setEnabled(true);
```

The template mesh itself must be a `BABYLON.Mesh` (not `AbstractMesh`) for `createInstance()` to be available. If the GLB root is a `TransformNode`, call `createInstance()` on the first child mesh instead.

10.2 Texture Atlasing for Furniture

Even with instancing, each unique material still requires its own draw call per instance if materials differ. With six default furniture types each carrying a distinct material, this can add up quickly in rooms with dense decorating. The solution is a **texture atlas**: a single large texture (2048×2048) that contains all furniture textures in non-overlapping UV regions. All furniture meshes share one material with this atlas, reducing the material draw call count from N to 1 regardless of how many furniture types are in the room.

Creating a texture atlas in Blender:

80. Open all furniture objects in one Blender scene simultaneously.
81. Select all (A), enter Edit Mode, select all faces (A).
82. Open the UV Editor. Go to **UV → Pack Islands** with a margin of 0.005. This auto-arranges all UV islands from all furniture into the 0–1 UV space without overlap.
83. Bake all materials onto a single 2048×2048 image texture using Blender's **Cycles Bake** (Render Properties → Bake → Bake Type: Diffuse, Contributions: Color only).
84. Replace all furniture materials with a single material using the baked atlas as Base Color.
85. Re-export all furniture GLBs. They will now share the same material name and Babylon.js will create a single material for all of them.

This is an optimization step for after the art is finalized in Phase 2. The placeholder GLBs from Section 7.3 already use a single flat color per piece, so texture atlasing is primarily relevant when you introduce painted/textured furniture art in later phases.

10.3 Level of Detail (LOD) Strategy

Babylon.js has built-in LOD support through `mesh.addLODLevel()`. A LOD level specifies a distance threshold beyond which a simpler mesh (or no mesh) is substituted for the full-quality mesh. For HavenWorld's 20×20-meter rooms this is most impactful for remote player avatars and background decorative props:

```
// Remote player avatar LOD setup // avatarMeshHigh: full 8,000-triangle mesh
// avatarMeshLow: simplified 1,500-triangle mesh (created in Blender with
Decimate modifier) avatarMeshHigh.addLODLevel(8, avatarMeshLow); // Switch
at 8 meters avatarMeshHigh.addLODLevel(18, null); // Fully hide
beyond 18 meters // Decorative prop LOD propMesh.addLODLevel(12, null); //
Hide distant props beyond 12m
```

To generate the low-poly avatar mesh in Blender: duplicate the body mesh, apply a **Decimate modifier** at 0.18 ratio (reduces ~8,000 tris to ~1,440 tris), apply the modifier, export as a separate `base_avatar_lod.glb`. Load this alongside the main avatar GLB and use it for LOD switching.

10.4 Render Loop Performance Budget

The following table shows HavenWorld's target performance budget for a fully populated personal room with two players:

Metric	Target	Acceptable	Action Required If Exceeded
Frame rate (modern laptop)	60 FPS	45 FPS	Enable LOD, reduce max room occupancy
Frame rate (mid-range mobile)	30 FPS	24 FPS	Reduce polygon budgets 50%
Frame time per render call	<16.7ms	<22ms	Profile draw calls in Babylon Inspector
Total scene triangles	<200,000	<300,000	Audit room and furniture budgets
JS heap memory	<200MB	<300MB	Dispose unused assets, check leaks

Metric	Target	Acceptable	Action Required If Exceeded
Simultaneous players (per room)	12	8	Increase server MAX_ROOM_OCCUPANCY cap
GLB load time on WiFi	<2 seconds	<5 seconds	Compress textures, reduce file size

Babylon.js provides two useful engine-level optimizations applicable once a room is fully loaded and geometry is static:

```
// After room loading is complete: scene.freezeActiveMeshes(); // Skip
frustum culling calculations each frame scene.autoClearDepthAndStencil =
false; // Skip redundant clears (safe when no transparency overlap) // For
mobile / lower-end devices, detect and adapt: const engine =
scene.getEngine(); if (engine.getFps() < 30) { // Switch to aggressive
performance mode engine.setHardwareScalingLevel(1.5); // Render at 66%
resolution scene.pointerMovePredicate = () => false; // Disable per-frame
picking }
```

11. Complete Phase 2 Testing Checklist

Run this checklist top-to-bottom after completing each section. A failing item blocks further testing of items that depend on it. Fix failures before moving forward.

3D Room Loading

- Town Square GLB loads without any console errors (no red messages in DevTools).
- Floor is walkable — clicking anywhere on the floor triggers the walk animation and the avatar moves to that point.
- `navmesh_floor` mesh is present in the Babylon Inspector but not visible in the game view (opacity = 0 or `setEnabled(false)`).
- Room GLB loads in under 3 seconds on a WiFi connection (measure with DevTools Network tab).
- Babylon Inspector (Ctrl+I) shows all mesh names exactly matching Section 2.2 naming rules.

- Room walls correctly block camera movement (if camera collision is configured).

Avatar System

- `base_avatar.glb` loads with armature visible in the Babylon Inspector's Skeleton tab.
- Babylon Inspector shows exactly 5 AnimationGroups: `idle`, `walk`, `sit`, `wave`, `dance`.
- Idle animation plays continuously on spawn with smooth looping (no frame-skip at loop point).
- Clicking the floor triggers walk animation during movement; idle resumes within 0.3 seconds of arrival.
- Name tag floats above the avatar's head at the correct height and always faces the camera.
- A second browser tab logged in as a different test user shows that user's avatar as a separate `RemoteAvatar` instance.
- Remote avatars play the correct animation state (idle when standing, walk when moving).

Avatar Customizer

- Clicking "Edit Avatar" opens the customizer overlay without any JavaScript exceptions.
- The live 3D preview canvas initializes and renders the avatar within 2 seconds of the overlay opening.
- Skin tone swatches visually update the avatar in the live preview within 100ms of clicking.
- All three body sliders (Body Type, Height, Build) cause visible morph target deformation in the preview.
- "Save" button sends a PUT to `/api/users/me/avatar` and returns HTTP 200.

- After saving, other players in the room see the updated avatar within 2 seconds.
- Refreshing the page after saving loads the customized avatar correctly from the server.
- “Cancel” reverts the preview to the last saved state and closes the overlay without saving.

Furniture System

- All 6 default furniture items are present in the database after re-running the seed.
- New user registrations automatically receive all `isDefault: true` items in their inventory.
- Furniture items placed in the database load in the room at the correct position and rotation.
- Babylon Inspector shows instanced meshes (InstancedMesh type) when more than one of the same furniture type is placed.
- `GET /api/rooms/{roomId}/furniture` returns HTTP 200 with a correctly shaped JSON array.

Personal Room Editor

- “Decorate” button is visible only when the logged-in user is viewing their own room.
- Clicking “Decorate” shows the grid overlay and inventory panel without console errors.
- Clicking a furniture item in the inventory panel activates a ghost mesh that follows the mouse on the floor.
- Ghost mesh position snaps to 0.5-meter grid increments (verify by watching coordinates in Inspector).
- Left-clicking places furniture at the ghost position and the ghost mesh is disposed cleanly.

- Right-clicking placed furniture opens the context menu with Move, Rotate 90°, and Remove options.
- Rotate 90° rotates the selected piece exactly 90 degrees ($\pi/2$ radians) on the Y axis.
- Remove deletes the piece from the scene immediately.
- “Save Layout” sends a POST to `/api/rooms/{roomId}/furniture/layout` and receives HTTP 200.
- After saving, other players in the room see the updated layout within 2 seconds (real-time socket sync).
- “Cancel” reloads the original layout from the server and exits edit mode.

Performance

- Open Chrome DevTools → Performance, record 5 seconds with 2 players in a room. Average frame time under 16.7ms.
- Memory usage in the Performance tab stays under 256MB.
- Enter and exit the room 5 times in sequence. Memory does not increase each cycle (no leaks).
- Babylon.js Inspector → Statistics shows total active vertices consistent with the polygon budget in Section 10.4.

12. Git Workflow and Deployment

12.1 Committing Phase 2

Use the following sequence of granular commits. Each commit message follows the **Conventional Commits** format used throughout the HavenWorld series. Granular commits make it easier to bisect regressions later and produce a clean changelog for release notes.

```
## Stage and commit the Babylon.js engine wrappers git add  
apps/client/src/engine/ git commit -m "feat(client): production HavenEngine,  
SceneManager, RoomLoader with GLB support" ## Avatar system git add
```

```
apps/client/src/world/AvatarController.ts git add
apps/client/src/world/RemoteAvatar.ts git commit -m "feat(client):
AvatarController with GLB loading, morph targets, animation blending" ##
Furniture system git add apps/client/src/world/FurnitureManager.ts git add
apps/client/src/world/RoomEditor.ts git commit -m "feat(client):
FurnitureManager with instancing; RoomEditor with grid snap and context menu"
## UI layer git add apps/client/src/ui/AvatarCustomizer.ts git commit -m
"feat(client): 3D avatar customizer UI with live preview, morph sliders,
color pickers" ## 3D assets (GLB files) git add public/assets/ git commit -m
"assets: add base_avatar.glb, town_square.glb, and placeholder furniture
GLBs" ## Server routes and socket handlers git add apps/server/src/routes/
git add apps/server/src/socket.ts git commit -m "feat(server): avatar
customization route, furniture layout route, socket sync handlers" ## Shared
package git add packages/shared/src/ git commit -m "feat(shared): add 3D
avatar and furniture socket event constants, AvatarData type" ## Database
seed git add prisma/seed.ts git commit -m "feat(db): seed 10 furniture items
with isDefault flag and inventory grants" ## Tag the Phase 2 release git tag
v0.2.0-3d-phase git push origin main --tags
```

12.2 Cloudflare Pages Deployment Check

After pushing to `main`, navigate to your Cloudflare Pages dashboard and monitor the build log for the triggered deployment. The build should complete without TypeScript errors. Key things to verify once the build finishes:

86. The Cloudflare build log shows **0 TypeScript errors** and exits with code 0.

87. In the deployed URL (`https://havenworld.pages.dev`), open DevTools →

Network → filter by **glb**. Loading the game lobby should trigger requests for `base_avatar.glb` and the room GLB, both returning **HTTP 200**.

88. Filter Network by **wasm**. If Havok physics is imported, verify

`HavokPhysics.js.wasm` returns HTTP 200. If you removed Havok imports for Phase 2 (recommended since physics is a Phase 3 feature), this request should not appear.

89. Open the browser console. No red errors should appear during normal gameplay. Yellow warnings about missing optional features are acceptable.

90. Verify the Cloudflare Pages **asset headers**: GLB files should have Content-Type: `model/gltf-binary`. If they show `application/octet-stream`, add a MIME type override in your `_headers` file:

```
# public/_headers /assets/*.glb Content-Type: model/gltf-binary Cache-
Control: public, max-age=2592000, immutable
```


12.3 Oracle VM Server Updates

[LINUX] SSH into the Oracle VM and deploy the server-side changes:

```
ssh ubuntu@[your-oracle-ip] cd ~/HavenWorld git pull origin main # Install
any new server dependencies pnpm install --filter server # Apply database
migrations (if any new ones were added) pnpm --filter server prisma migrate
deploy # Run the updated seed script pnpm --filter server prisma db seed #
Restart the server process via PM2 pm2 restart havenworld-server # Verify
startup logs pm2 logs havenworld-server --lines 50
```

The server logs should show these lines (in order) after restart, with no error lines in between:

91.Prisma Client connected to PostgreSQL

92.Express server listening on port 3001

93.Socket.io ready – CORS origins: [...]

94.Room registry initialized

If you see `ERROR: relation "room_furniture" does not exist`, the migration was not applied. Run `pnpm --filter server prisma migrate deploy` again and check for migration file conflicts in the `prisma/migrations/` directory.

13. Troubleshooting Phase 2B

The following are the eight most common issues encountered when completing Phase 2B for the first time, with exact root causes and solutions.

#	Error / Symptom	Root Cause	Solution
1	Cannot read properties of undefined (reading 'meshes')	GLB file failed to load. <code>SceneLoader.ImportMeshAsync</code> rejected the promise silently or the path is wrong.	Check the file path passed to <code>ImportMeshAsync</code> . The folder argument must end with <code>/</code> and the file argument must be just the filename. Verify the file exists by dragging it to <code>sandbox.babylonjs.com</code> . Check the

#	Error / Symptom	Root Cause	Solution
			Network tab for a 404 on the GLB request.
2	AnimationGroup not found: walk (or any animation name)	Animation group names in the GLB do not match the string keys in <code>initAnimations()</code> .	Open the GLB in the Babylon.js sandbox. Expand Animation Groups in the Inspector and read the exact names as exported by Blender. Update the <code>validNames</code> array in <code>initAnimations()</code> to match exactly (case-sensitive).
3	Morph target sliders have no visual effect on the avatar	Shape key names in Blender do not match the switch-case strings in <code>applyMorphTargets()</code> , or Shape Keys were not enabled in the GLB export settings.	In the Babylon Inspector, select the body mesh → click the Morph Targets icon. Verify target names are lowercase and match exactly. If no morph targets appear, re-export the GLB with Shape Keys = ON in the Blender export dialog.
4	Placed furniture sinks into the floor or floats above it	The y coordinate stored in <code>room_furniture</code> does not account for the mesh's pivot being at its center rather than its bottom.	In <code>RoomEditor.snapToGrid()</code> , calculate <code>snappedY</code> as <code>floorY + ghostMesh.getBoundingBox().extendSize.y</code> . This adds half the bounding box height to the floor level, placing the bottom of the mesh exactly on the floor.
5	Ghost mesh	<code>clearGhostMesh()</code>	Ensure

#	Error / Symptom	Root Cause	Solution
	remains visible in the scene after placing furniture	is not being called after placement, or the ghost mesh reference is not being nulled correctly.	<code>this.ghostMesh.dispose()</code> is called followed by <code>this.ghostMesh = null</code> in <code>clearGhostMesh()</code> . Verify <code>clearGhostMesh()</code> is called at the end of <code>placeFurnitureFromGhost()</code> .
6	CORS error on PUT <code>/api/users/me/avatar</code>	The new route is not covered by the Express CORS configuration, or the route is not under the <code>/api</code> prefix handled by the CORS middleware.	Verify the CORS middleware in <code>apps/server/src/index.ts</code> is applied before route registration and covers all <code>/api/*</code> paths. Check that the route is registered as <code>router.put('/me/avatar'...)</code> inside the <code>usersRouter</code> which is mounted at <code>/api/users</code> .
7	Havok WASM loads but physics does not work / COOP header error	Havok Physics requires Cross-Origin Opener Policy (COOP) and Cross-Origin Embedder Policy (COEP) headers, which are not set in Cloudflare Pages by default.	If physics is not needed in Phase 2, remove <code>@babylonjs/havok</code> from all imports until Phase 3. For Phase 2, all furniture movement is positional (no physics simulation). If you need physics later, add COOP/COEP headers to <code>public/_headers</code> .
8	Second player's avatar shows wrong colors after another player saves	The <code>RemoteAvatar</code> receives the <code>avatar:update</code> socket event but does not call	In the socket event handler for <code>SOCKET_EVENTS.AVATAR_UPDATE</code> , verify the payload

#	Error / Symptom	Root Cause	Solution
	customization	applyCustomization() with the new data.	contains the full AvatarData object and that remoteAvatar.applyCustomization(payload.avatarData) is called. Log the payload to confirm the data is present.

14. What's Next: Phase 3 Preview

With Phase 2 complete, HavenWorld now has a fully functioning 3D browser world: players can enter rooms represented by real GLB assets, walk around as rigged humanoid avatars with five animations, customize their appearance with morph targets and color pickers, and decorate their personal rooms with furniture they can place, rotate, and save in real time. The architectural foundation — the Babylon.js engine wrapper, the socket sync layer, the Prisma-backed REST API — is solid enough to support the social and economy features of Phase 3.

Part 6 (Phase 3) will add:

- **Economy System** — In-game currency called **Haven Coins**, a shop interface that pulls from the `items` table, a purchase endpoint with idempotent transaction logging in a new `transactions` table, and a transaction history view in the profile page. All currency is virtual; no real-money transactions. GPL-3.0 compliant.
- **Live Events** — A scheduled event system using a new `events` table. Public rooms can be configured as temporary event venues with an event banner, countdown timer, and elevated player capacity for the duration. Uses the existing room system with an `eventRoomType` field.
- **Friends System** — The `friends` table from Part 4B gets its UI: send friend requests, accept/decline, view online friends, see which room a friend is in,

and one-click visit via a teleport that changes your current room socket subscription.

- **Notification System** — Browser Notification API integration (with permission request on login) for friend requests, new items, and event reminders. In-game toast notifications for real-time events using a `ToastManager` component.
- **Mobile Touch Controls** — Babylon.js has strong touch event support built in. Phase 3 adds a virtual joystick for movement, tap-to-walk on mobile, and pinch-to-zoom camera control. A `DeviceDetector` utility switches the control scheme automatically.
- **Performance Profiling** — A dedicated profiling session using Chrome DevTools Performance, the Babylon.js Inspector frame graph, and custom performance marks. Documented hot spots and fixes for scenes with 12 simultaneous players.
- **Moderation Enhancements** — A report-a-player UI (right-click avatar → Report), admin dashboard to see reports, and a server-side kick command that closes the target player's socket connection and locks them from re-joining a room for a cooldown period.
- **Analytics (Self-Hosted)** — A `daily_active_users` table populated by a nightly cron job on the Oracle VM. A simple admin chart page showing DAU, room popularity, and economy throughput. No external analytics service; 100% self-hosted, no paid tier.

Appendix: Quick Reference Cards

A. Time Estimate

Total estimated development time for Phase 2: 3–5 weekends for a solo developer already comfortable with TypeScript. Blender pipeline work (Sections 2–4) accounts for roughly 40% of the total time on first pass. The TypeScript

implementation (Sections 5–8) is faster once assets are available. Testing and iteration (Section 11) accounts for approximately 20% of total time.

B. New npm Packages Added in Phase 2

npm / pnpm Dependencies

Package

Workspace

Version (approx)

Purpose

@babylonjs/core

client

^7.x

Core 3D engine (already from Part 5A)

@babylonjs/loaders

client

^7.x

glTF 2.0 / GLB loader plugin

@babylonjs/gui

client

^7.x

In-world GUI: name tags, HUD elements

@babylonjs/materials

client

^7.x

GridMaterial for room editor floor grid

@gltf-transform/cli

dev (global)

^4.x

GLB inspection and texture compression CLI

zod

server

^3.x

Runtime validation of AvatarData and layout payloads (already from Part 4A)

C. New Files Created in Phase 2

New Source Files

File Path

Purpose

apps/client/src/world/AvatarController.ts

Local player avatar: GLB loading, animations, morph targets, name tag, movement

apps/client/src/world/FurnitureManager.ts

Room furniture: fetch, load, instance, pick

apps/client/src/world/RoomEditor.ts

Personal room editor: ghost mesh, grid snap, place/rotate/remove, save

apps/client/src/ui/AvatarCustomizer.ts

Full-screen avatar customizer overlay UI

public/assets/avatars/base_avatar.glb

Rigged humanoid avatar with morph targets and 5 animations

public/assets/rooms/town_square.glb

First public room: 20×20m plaza with walls and props

public/assets/furniture/chair.glb

Placeholder wooden chair furniture

public/assets/furniture/table.glb

Placeholder round table furniture

public/assets/furniture/sofa.glb

Placeholder blue sofa furniture

public/assets/furniture/lamp.glb

Placeholder floor lamp furniture

public/assets/furniture/bookshelf.glb

Placeholder bookshelf furniture

public/assets/furniture/rug.glb

Placeholder red rug furniture

source_assets/mixamo_ybot_tpose.fbx

Raw Mixamo Y Bot FBX (source file, not deployed)

source_assets/anim/idle.fbx

Mixamo idle animation (source file, not deployed)

source_assets/anim/walk.fbx

Mixamo walk animation (source file, not deployed)

source_assets/anim/sit.fbx

Mixamo sit animation (source file, not deployed)

source_assets/anim/wave.fbx

Mixamo wave animation (source file, not deployed)

source_assets/anim/dance.fbx

Mixamo dance animation (source file, not deployed)

public/_headers

Cloudflare Pages response headers: GLB MIME type and cache control

D. Server Endpoint Changes

API Endpoint Changes in Phase 2

Method

Path

Auth

Description

PUT

/api/users/me/avatar

Required

Save avatar customization. Broadcasts

avatar:update

to current room.

GET

/api/rooms/:roomId/furniture

Required

Fetch placed furniture list for a room. Returns array of

FurniturePlacementData

.

POST

/api/rooms/:roomId/furniture/layout

Required (owner)

Atomically replace entire room furniture layout. Broadcasts

room:furniture_updated

.

GET

/api/users/me/inventory

Required

Fetch authenticated user

,

s inventory. Supports

?type=furniture

filter.

E. New Socket Events

Socket.io Events Added in Phase 2

Event Constant

String Value

Direction

Payload

AVATAR_UPDATE

avatar:update

Server → Room

{ userId, avatarData: AvatarData }

ROOM_FURNITURE_UPDATED

room:furniture_updated

Server → Room

{ roomId, action?, item?, furnitureId? }

FURNITURE_PLACE

furniture:place

Client → Server

{ roomId, placement: FurniturePlacement }

FURNITURE_REMOVE

furniture:remove

Client → Server

{ roomId, furnitureId: string }

F. Final Release Tag

```
git tag v0.2.0-3d-phase git push origin main --tags
```

You've completed Phase 2.

HavenWorld now has a production-quality 3D pipeline with rigged avatars, Mixamo animations, in-game avatar customization, a furniture system with GPU instancing, and a real-time room editor. All systems are wired end-to-end from Blender to PostgreSQL to the browser. Push your commits, tag v0.2.0-3d-phase, and get ready for Phase 3: Economy, Friends, Events, and Mobile.